

Dossier de projet **Passage diplôme CDA**

Alexandre

Sommaire

I. Présentation du projet	5
1. Contexte de création	5
2. Besoins et objectifs	6
II. Gestion de projet	7
1. Contraintes et risques du projet	7
2. Périmètre et limites du projet	8
3. Méthodologie de travail	10
4. Environnement de travail	12
A. Technologies utilisées	12
a. Planification	12
b. Création d'assets	12
c. Communication	13
d. Développement	13
5. Organisation de projet	15
A. Gestion du temps, moyens, budget	15
B. Planning	18
6. Organisation de développement	19
7. Suivi quotidien et Daily	20
8. Gestion des priorités	20
9. Gestion du versionnement GitHub	22
10. Gestion des versions et Releases	23
III. Conception et maquettage	25
1. Parcours utilisateur	25
2. Maquettes et conception des interfaces	26
3. Interfaces principales	27
4. Choix ergonomiques	31
5. Accessibilité et lisibilité	32
IV. Architecture logicielle	33
1. Choix technologiques	33
2. Architecture générale	35
3. Architecture Client / Serveur	36
A. Client / FrontEnd	36
B. Serveur / BackEnd	36
4. Architecture du FrontEnd	37

5. Architecture du BackEnd -----	37
A. Architecture multicouche ·····	38
6. Projet Shared -----	38
7. Communication FrontEnd / BackEnd -----	40
A. Flux d'une requête ·····	40
V. Conception et persistance des données -----	41
1. Besoins en données -----	41
2. MCD -----	41
3. MLD -----	42
4. Modèle physique -----	42
5. Entity Framework Core et DbContext -----	43
6. Persistance des données -----	45
7. Sauvegarde / restauration -----	45
VI. Développement des composants -----	46
1. Organisation du code -----	46
2. Composants FrontEnd -----	47
3. Composants BackEnd -----	48
4. Communication avec l'API BackEnd -----	49
5. Exemple complet d'une fonctionnalité -----	49
VII. Sécurité -----	51
1. Principes de sécurité -----	51
2. Authentification -----	51
3. Protection des données sensibles -----	52
A. Hachage des mots de passe ·····	52
4. JWT et authentification des requêtes -----	54
5. Validation et protection des données reçues -----	55
A. Validation côté client ·····	55
B. Validation côté serveur ·····	55
C. Protection contre les injections ·····	55
6. Rate Limiting -----	56
7. Suppression des données utilisateur / RGPD -----	56
VIII. Tests et validation -----	57
1. Stratégie et environnement de test -----	57
2. Tests fonctionnels et techniques -----	57
3. Tests de régression et validation utilisateur -----	58

4. Résultats et limites -----	59
IX. Déploiement et DevOps -----	60
1. Environnements Development / Production -----	60
2. Préparation et déploiement du BackEnd -----	60
3. Builds et Releases -----	62
4. Processus de mise en production et limites DevOps -----	63
X. Bilan de projet -----	64
XI. Annexe -----	65
1. Script Commands.sql -----	65
2. Script DataBase.sqlite -----	67
3. Script ServerRequestManager -----	68
4. Script Authenticator, méthode OnSendButtonPressed() -----	70
5. Script UserProgressionController, security checks -----	71
6. Script Program, rate limiter setup -----	72

I. Présentation du projet

1. Contexte de création

Je suis actuellement en cours d'obtention du permis de conduire.

J'ai pu constater durant mes heures de conduite, que j'avais encore beaucoup de mal à rapidement savoir où était ma gauche ou ma droite.

Ça a donné lieu à quelques moments dangereux sur la route, où je me suis dirigé dans la mauvaise direction, car j'avais confondu ma gauche et ma droite.

Habituellement, j'ai un moyen mnémotechnique pour savoir où est la gauche et la droite. J'imagine une carte de l'Europe et me dit qu'à gauche c'est l'Ouest et à droite l'Est.

Cependant cette stratégie prend trop de temps par rapport aux standards de la route.

Par conséquent depuis quelques temps, j'essaie de m'entraîner à très rapidement identifier ma gauche et ma droite. Cependant ce n'est pas encore suffisant.

Au vu de mon futur passage de l'épreuve de CDA (Concepteur Développeur d'Application), j'ai décidé de mêler l'utile à l'agréable.

J'ai décidé de créer une application validant les compétences requises pour l'obtention de mon diplôme CDA, qui, en plus, me permettrait de régler mon problème de gauche et de droite.

L'application aura pour but d'apprendre à très rapidement identifier la gauche et la droite.

L'application se présente sous la forme d'un jeu dynamique composé de différents niveaux. Plus le joueur progresse dans les niveaux, plus le temps disponible pour identifier la gauche ou la droite est réduit.

Selon sa rapidité, le joueur obtient un score ainsi qu'une évaluation sous la forme d'une note à trois étoiles.

Pour répondre au demande du CDA, l'application aura obligatoirement un système de création et de connexion de compte.

La progression du joueur sera alors sauvegardée dans une base de données.

2. Besoins et objectifs

Le projet répond à deux objectifs distincts mais complémentaires :

1. Le premier objectif est de répondre à un besoin personnel.
Comme expliqué précédemment, j'éprouve des difficultés à identifier rapidement ma gauche et ma droite, notamment dans des situations où une réponse rapide est nécessaire (conduite).

L'application a donc pour objectif de me permettre de m'entraîner à identifier rapidement une direction, en augmentant progressivement la difficulté des exercices. Le joueur doit être capable de répondre à une succession de directions dans un temps limité. Ses performances sont ensuite évaluées afin de lui permettre de mesurer sa progression.
2. Le second objectif est lié à ma formation de Concepteur Développeur d'Applications (CDA). J'ai volontairement choisi de transformer ce besoin personnel en un projet suffisamment complet pour mettre en œuvre et démontrer les différentes compétences attendues dans le cadre de l'obtention de ce diplôme.

Le projet ne se limite donc pas à la création d'un jeu fonctionnel.

Il comprend également une architecture séparant le FrontEnd (Client) du BackEnd (Server), une API REST permettant leur communication, une base de données relationnelle permettant de conserver les données des utilisateurs, un système d'authentification, ainsi que diverses mesures de sécurité pour limiter d'éventuels actes malveillants.

Ces deux objectifs, ainsi que la contrainte de temps, ont ainsi guidé mes choix durant le développement.

L'objectif était de créer une application suffisamment simple pour rester réalisable dans le temps disponible, tout en étant suffisamment complète pour me permettre de démontrer les compétences attendues en vue de l'obtention du diplôme CDA.

II. Gestion de projet

1. Contraintes et risques du projet

Le projet étant réalisé dans un contexte contraint, plusieurs risques susceptibles d'affecter son développement ont été identifiés dès le début.

Leur identification permettait d'anticiper leurs conséquences et de mettre en place des mesures permettant de les limiter.

Contrainte / risque	Conséquences	Solution
Projet non principalement orienté sur le sujet de mes études : développement de jeux vidéo	Certaines des compétences nécessaires et demandées, ne sont pas maîtrisées	Apprendre les compétences nécessaires au projet, ainsi qu'au diplôme
Manquer de temps pour produire le projet et créer les documents demandés	Ne pas pouvoir rendre les livrables avant la date butoir → Repasser l'épreuve	Viser à créer un projet qui permet de rapidement atteindre les objectifs fixés + Ainsi qu'allouer une période suffisante pour atteindre ces objectifs
Projet réalisé seul	L'intégralité du projet devra être conçu et produit par moi-même	Accepter cette contrainte et en faire une opportunité pour développer toutes ses compétences de conception et développement d'application
Risque de régression lors du développement	Une nouvelle fonctionnalité peut casser une fonctionnalité existante	Avoir un système de branches GitHub dédiées + tests avant intégration
Risque de ne pas disposer d'une version stable à la fin du projet	Devoir présenter une version incomplète ou instable	Avoir un MVP le plus tôt possible + branche GitHub « main » stable + Releases GitHub

L'une des priorités les plus importantes du projet est d'avoir un périmètre (ou scope en Anglais) le plus petit possible, tout en me permettant de produire un projet qui remplit les objectifs fixés.

2. Périmètre et limites du projet

Lors de la création d'un projet, il est important de définir rapidement son périmètre (ou scope). Il permet de cadrer le projet, de déterminer précisément les fonctionnalités qui doivent être réalisées, les objectifs à atteindre, ainsi que les éléments qui n'en font volontairement pas partie.

Déterminer un périmètre permet donc d'adapter son projet à son temps, moyen et budget disponible.

Le respect de ce périmètre permet également de limiter les risques de dérive des objectifs du projet (ou scope creep) au cours du développement.

Cette définition est particulièrement importante dans le cadre de ce projet, puisque son développement est réalisé dans une période de temps limitée.

Il est donc nécessaire de conserver un périmètre suffisamment restreint pour permettre de terminer le projet dans les délais, tout en conservant suffisamment de fonctionnalités pour répondre aux objectifs fixés.

J'ai donc analysé les besoins que j'ai, et les ai transformés en besoins fonctionnels et non fonctionnels :

Fonctionnalités incluses dans le projet (liste non-exhaustive) :

Fonctionnalités principales :

- la création, connexion et suppression d'un compte utilisateur
- plusieurs niveaux de difficulté pour les niveaux du jeu
- un système de réponse basé sur l'identification de la gauche et de la droite
- un système de temps maximal de réponse évoluant selon le niveau
- l'obtention d'un score en fonction des performances du joueur dans chaque niveau
- un système d'évaluation sous forme d'étoiles (conversion score → étoiles)
- un système de pause permettant au joueur de suspendre une partie

Fonctionnalités techniques / contraintes intégrées au périmètre :

- API REST entre le FrontEnd et le BackEnd
- un système d'authentification permettant d'identifier les différents utilisateurs
- une base de données relationnelle permettant de sauvegarder et conserver les données nécessaires au fonctionnement de l'application ainsi que la progression du joueur
- des mécanismes de sécurité permettant de contrôler les données envoyées au serveur

Éléments volontairement exclus du projet (liste non-exhaustive) :

Afin de conserver un périmètre adapté au temps disponible et aux objectifs du projet, certaines fonctionnalités ont volontairement été exclues.

Le projet ne prévoit notamment pas :

- la réalisation de graphismes complexes, l'objectif étant de privilégier la fonctionnalité et la lisibilité de l'application
- le déploiement sur une infrastructure distante
- un mode multijoueur versus
- de classement en ligne entre les joueurs

Ces éléments pourraient être envisagés comme des évolutions futures, mais ne sont pas nécessaires pour répondre aux objectifs actuels du projet.

Le périmètre défini a ainsi pour objectif de trouver un équilibre entre les besoins fonctionnels de l'application, les compétences que je souhaite mettre en œuvre dans le cadre du CDA et les contraintes de temps liées au développement du projet.

3. Méthodologie de travail

J'ai décidé de ne pas utiliser une méthode Agile formelle telle que Scrum avec l'ensemble de ses principes.

J'ai cependant décidé d'adopter une organisation inspirée des méthodes Agiles, que j'ai adaptée à la taille du projet, à sa durée et au fait que je sois la seule personne à travailler sur ce dernier.

L'objectif était de conserver les principaux avantages d'une organisation Agile :

- développement progressif
- adaptation des priorités
- retours réguliers

Tout en évitant de consacrer une part trop importante du temps disponible à des tâches de gestion qui n'auraient pas rendu la production du projet plus efficace.

Le développement a ainsi été réalisé progressivement par fonctionnalités.

Une fonctionnalité était alors, développée, testée, puis intégrée au reste du projet une fois considérée comme suffisamment stable.

Cette organisation permettait également de modifier les priorités au cours du développement.

La priorité d'une fonctionnalité pouvait donc être ajustée en fonction de l'état du projet, des difficultés rencontrées ou des retours obtenus lors des tests.

La méthodologie utilisée peut donc être résumée comme une organisation Agile adaptée à un projet individuel, avec des cycles courts de développement et de validation plutôt qu'une méthodologie plus lourde et coûteuse en temps de gestion.

Le développement était donc organisé en cycles successifs. Chaque cycle correspondait au développement d'une mécanique ou d'un système du projet.

Une version du projet est la somme de plusieurs cycles.

Lorsqu'une version du projet est terminée, une période de test par des personnes extérieures au projet est effectuée (explication en détail dans la section « [Déploiement et DevOps](#) »)

Un Daily est utilisé afin de noter les tâches prioritaires à effectuer, ainsi que suivre l'avancement du projet au fil des jours.

Le dépôt GitHub du projet permet aussi de suivre l'avancement de ce dernier.

Exemple d'un jour de Daily

(L'objectif n'est pas de vous montrer le contenu d'un Daily, mais plutôt de vous montrer sa forme)

2. Mardi 07/07/2026 :

Matin :

- Finir le rendu visuel des UIs dans la Scène Level
 - Implémentation complète de la bar qui affiche le score et les étoiles (dans le menu Game over) + commits :

• Implementing ScoreBar Script (check desc) ↗

renders showing to the player the score he made during the Level, and how many stars he achieved.

• Creating new Prefab: ScoreBar ↗

↳ Making GameOver Script use ScoreBar Script ↗

• Making GameOver Prefab use ScoreBar Prefab ↗

- Implémentation de l'affichage dans le menu GameOver d'un feedback pour savoir si on a débloqué le prochain niveau + ajout d'un bouton pour passer au niveau suivant sans avoir à passer par le menu de sélection de niveau + commits :

• Implementing NextLevelButton Script ↗

• Creating new Prefab: NextLevelButton ↗

⚡ Modifying OnLevelWontEvent + OnLevelLostEvent Events (check desc) ↗

Adding a new parameter: It tells if the next level is already unlocked.

• Adding the possibility for the LevelManager to start the next Level ↗

• Updating LevelManager to send the right data when the Level ends ↗

• Implementing the NextLevelButton and LevelManager changes ↗

• Making GameOver Prefab use NextLevelPrefab Prefab ↗

Prise de pause manger à : 13h40

Après-midi :

- Implémentation d'un compte à rebours à vitesse variable (1s = temps de réponse maximal entre réponse) qui se lance avant le lancement d'une partie + commits :

• Adding the Assets/UIs/Level/Countdown folder ↗

• Implementing the LevelCountdown Script ↗

• Creating new Prefab: Countdown ↗

↳ Making LevelManager Script use LevelCountdown Script ↗

• Adding a LevelCountdown instance to Level Scene ↗

- Implémentation de l'affichage d'un petit tuto / texte pour dire qu'on peut utiliser les touches QD et les flèches directionnel pour répondre à une direction.
S'affiche dans LevelSelection si le joueur n'a pas le Level 2 débloqué + commits :

• Adding the Assets/UIs/ShortcutTutorial folder ↗

• Implementing the ShortcutTutorial Script (check desc) ↗

Only handles showing the ShortcutTutorial UI if the player hasn't unlocked a specific level.

• Creating new Prefab: ShortcutTutorial ↗

• Adding a ShortcutTutorial to LevelSelection Scene (check desc) ↗

It also adds a button that allows the player to show the ShortcutTutorial instances whenever he wants.

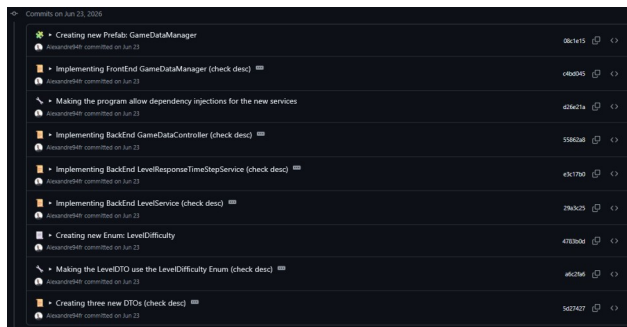
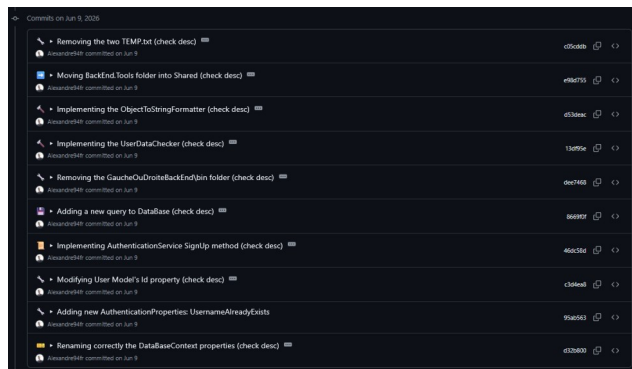
The button is accessible by using the Player dropdown.

Liste des tâches prioritaires à faire pour les jours de travail suivants :

- Potentiellement refaire le png du fog (car il n'a pas l'air très visible, blanc sur blanc)
Après attention il n'y a pas encore les chemins. Il faudrait sûrement déjà les faire avant de changer quoi que ce soit.
- Faire l'équilibrage du jeu
- Ajouter le système de pause

Exemple de commits effectués durant le projet

(L'objectif n'est pas de vous montrer le contenu des commits, mais plutôt de vous montrer leurs formes)



4. Environnement de travail

Afin de permettre ou d'accélérer la production du projet ou de ses documents pour ce projet. Divers outils et technologies ont été utilisés.

A. Technologies utilisées

a. Planification

Le pack LibreOffice et l'application Bloc-notes de Windows sont utilisés pour écrire tous les documents du projet.

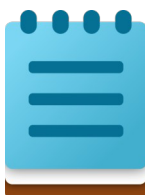
Cela comprend le Daily, qui permet d'assurer un suivi optimal de l'avancement du projet.

Figma, ainsi que Canva quant à eux sont utilisés pour la création de schémas explicatifs / mockups.

LibreOffice



Bloc-note



Figma



Canva



L'utilisation du Bloc-notes est préférée lorsqu'un document court doit être rédigé rapidement. L'utilisation du Bloc-notes plutôt que d'un outil plus complet comme LibreOffice, permet de se concentrer uniquement sur le contenu, et donc, de ne pas se faire distraire par la forme.

b. Création d'assets

Canva, mais principalement Gimp sont utilisés pour la création d'image / assets 2D.

Gimp



Canva



c. Communication

Malgré que je sois la seule personne à travailler sur la production du projet.

Le projet compte aussi des testeurs, il faut donc que je sois en capacité de leur parler, mais aussi de leur transmettre le projet à tester.

La communication avec les testeurs se fait via Discord.

L'envoi des versions du projet à tester se fait via SwissTransfer.

Les Releases, elles, se font via GitHub.

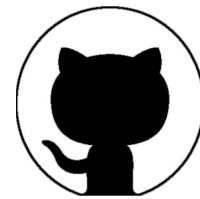
Discord



SwissTransfer



GitHub



d. Développement

Note : Toutes les raisons derrière le choix des technologies de développement seront développées dans la sous-section « [Choix technologiques](#) ».

Afin d'avoir un système de versionning pour le projet, GitHub et GitHub Desktop ont été utilisés.

GitHub



GitHub Desktop



J'ai choisi d'utiliser GitHub Desktop plutôt que Git, car GitHub Desktop possède une interface graphique permettant de mieux comprendre l'état du projet, et de manière générale, les actions qu'on effectue (commit, merge, changer de branche, etc...).

Ce choix a été rendu possible car le projet ne nécessite pas l'utilisation de commande Git complexe. GitHub Desktop peut donc effectuer toutes les commandes qu'un projet de cette taille a besoin.

J'ai pris la décision de formater mes commits et merge GitHub d'une certaine manière afin qu'il soit possible d'identifier rapidement la nature d'une modification sans avoir à consulter son contenu.

Cela comprend l'ajout d'emoji en préfixe des commits afin de rapidement pouvoir identifier le sujet de n'importe quel commit ou merge, exemple : 📄 ➤ Adding an example in "Dossier de projet" documentation

Une documentation servant de convention de nommage pour les commits et merge GitHub a donc été conçue :



Note : Ce document peut, et a été modifié durant le développement du projet afin de rester à jour et cohérent avec le projet.

5. Organisation de projet

A. Gestion du temps, moyens, budget

Une des tâches les plus importantes de la gestion de projet est de connaître :

- Le temps de travail disponible (permet d'avoir une vision long terme)
- Les moyens à disposition
- Le coût monétaire par mois et total du projet

Pour calculer et organiser cela, j'ai créé un grand tableau sous LibreOffice Calc :

Calcul manuel du temps total du projet :			
Nombre de jour total de travail :			75
Matin :		Après-midi :	
02:45:00	75	03:45:00	75
206:15:00		281:15:00	
487:30:00			

Calcul automatique du temps total du projet :			
Nombre de jour total de travail :			96
Date de début de projet :		Date de fin de projet :	
11/05/26		21/09/26	
Matin :		Après-midi :	
02:45:00	96	03:45:00	96
264:00:00		360:00:00	
624:00:00			
Note : Ce tableau automatique est utile pour avoir un aperçu rapide. Cependant, il sera incorrect s'il y a des trous dans l'emploi du temps. Dans ce cas, il vaut mieux passer par le tableau manuel (à gauche). C'est le tableau de gauche qui transmet les valeurs aux autres tableau du tableau.			

Calcul du temps des grands axes du projet :					
		Temps disponible par matin :	Temps disponible par après-midi :	Temps total disponible pour le projet :	Temps total disponible pour le projet (nombre) :
		02:45:00	03:45:00	487:30:00	487,50
		Matin disponible :	Après-midi disponible :		
		75	75		
		Valeurs respectées : Vert = Respecté Bleu = Inférieur Rouge = Supérieur	Tous les matin sont assignés : 75	Toutes les après-midi sont assignées : 75	Le temps total disponible est respecté : 487,50
Grand axes du projet :	Sous axes du projet :	Nombre de matin accordé :	Nombre d'après-midi accordé :	Temps disponible pour le sous axe (en nombre -> 42,75) :	Temps disponible pour le sous axe (en h et en m -> 42h45m) :
Pré-production	Création de toute l'administration / gestion du projet	1	1	6,50	6,30
	Création de toute l'administration / gestion du projet	3	3	19,50	19,30
	Temps d'imprévu	0	0	0,00	0,00
Production	Production de la Phase1 (MVP)	30	30	195,00	195,00
	Faire une période de QA (celle de la fin de la Phase1)	1	1	6,50	6,30
	Production de la Phase2 (Ajust de plus de feedback / finitions au projet)	5	5	32,50	32,30
	Faire une période de QA (celle de la fin de la Phase2)	3	3	19,50	19,30
	Temps d'imprévu	5	5	32,50	32,30
Post-production	Création des documents (Dossier de projet, Dossier professionnel)	17	17	110,50	110,30
	Création du diaporama	5	5	32,50	32,30
	Production de la Phase3 (Ajust de features supplémentaires)	0	0	0,00	0,00
	Temps d'imprévu	5	5	32,50	32,30

Il existe plusieurs feuilles de tableur, chacune avec son rôle.
Les précédentes images provenaient de la feuille de tableur « Feuille de calcul - Temps ».

⏪	⏩	+	Feuille de calcul - Temps	Temps - Gantt	Feuille de calcul - Temps tâches production	Feuille de calcul - Budget	Données sur les salaires	Feuille de calcul - Budget employés	Feuille de calcul - Budget frais généraux	Feuille de calcul - ANCIEN	
---	---	---	---------------------------	---------------	---	----------------------------	--------------------------	-------------------------------------	---	----------------------------	--

Données sur les salaires :

Jours ouvrés dans un mois			21,66	52 semaines × 5 jours ouvrés = 260 jours ouvrés 260 jours ouvrés par an ÷ 12 mois = 21,66			Ces valeurs sont indicatives et couvrent des contrats salariés classiques (CDI). En freelance, les TJM (Tarifs Journaliers Moyens) sont plus élevés, car le prestataire paie toutes ses charges : En général +30% à +50%.					
Poste	Brut Mensuel Junior (€)			Brut Mensuel Senior (€)			Brut Journalier Junior (€)			Brut Journalier Senior (€)		
	Minimum	Maximum	Moyen	Minimum	Maximum	Moyen	Minimum	Maximum	Moyen (arrondi)	Minimum	Maximum	Moyen (arrondi)
Producteur	2 400 €	2 600 €	2 500 €	3 900 €	4 350 €	4 125 €	110,80 €	120,04 €	115,42 €	180,06 €	200,83 €	190,45 €
Chef de projet	2 500 €	2 850 €	2 675 €	3 800 €	4 800 €	4 300 €	115,42 €	131,58 €	123,50 €	175,44 €	221,61 €	198,50 €
Game Designer	2 000 €	2 100 €	2 050 €	2 600 €	2 850 €	2 725 €	92,34 €	96,95 €	94,60 €	120,04 €	131,58 €	125,80 €
Lead Développeur	3 800 €	4 125 €	3 963 €	4 350 €	4 800 €	4 575 €	175,44 €	190,44 €	182,90 €	200,83 €	221,61 €	211,20 €
Développeur	2 400 €	2 600 €	2 500 €	2 850 €	3 050 €	2 950 €	110,80 €	120,04 €	115,40 €	131,58 €	140,81 €	136,20 €
Game Tester (QA)	1 625 €	1 950 €	1 788 €	2 200 €	2 400 €	2 300 €	75,02 €	90,03 €	82,50 €	101,57 €	110,80 €	106,20 €
Directeur artistique	3 500 €	4 500 €	4 000 €	5 000 €	6 500 €	5 750 €	161,59 €	207,76 €	184,70 €	230,84 €	300,09 €	265,50 €
Graphiste	2 175 €	2 400 €	2 288 €	2 725 €	2 925 €	2 825 €	100,42 €	110,80 €	105,60 €	125,81 €	135,04 €	130,40 €
Directeur sonore	2 950 €	3 150 €	3 050 €	3 900 €	4 350 €	4 125 €	136,20 €	145,43 €	140,80 €	180,06 €	200,83 €	190,40 €
Compositeur musical	2 400 €	2 600 €	2 500 €	3 050 €	3 500 €	3 275 €	110,80 €	120,04 €	115,40 €	140,81 €	161,59 €	151,20 €

Note : Pour le Producteur et Chef de projet, leur salaire n'est pas très haut comparé au Lead Dev et DA, et ce parce que le projet n'est pas AAA, sinon quel le salaire serait BEAUCOUP plus haut (+2000 / +4000)	Note : Si prit en Freelance, les créateurs de musiques et sons, coûtent beaucoup plus chère (200 / 400€ par jour)
---	---

Budget alloué aux frais salariaux :

Type d'emploi	Description	Multiplieur de coût en taxes	Multiplieur de coût de salaire brut (info)
CDI	Avec le profil actuel de l'entreprise (Auto-entreprise), les charges s'élèvent à 21 %, sinon c'est beaucoup plus.	21,00 %	0,00 %
Freelance	Les personnes en Freelance paient eux même leurs charges, faisant augmenter leur prix. Cependant cela veut dire que nous ne payerons pas de charges.	0,00 %	40,00 %
Stage	Si le stagiaire est payé 600€/mois alors il n'y a pas de taxe, SINON 25 % de taxe sont appliqués en moyenne. (Assurance maladie, vieillesse plafonnée, allocations familiales PAS de chômage, pas de retraite complémentaire, pas d'accident du travail)	25,00 %	-67,00 %
Alternance (Apprentissage)	Le multiplicateur de coût change selon l'âge de la personne, plus est elle âgée, plus l'employeur devra se rapprocher du SMIC (Payé : Accident du travail, Retraite complémentaire, Contribution formation).	6,50 %	-45,00 %
Alternance (Contrat pro)	Le multiplicateur de coût change selon l'âge de la personne, plus est elle âgée, plus l'employeur devra se rapprocher du SMIC (Allègement Fillon et exonérations spécifiques mais pas d'exonération totale)	20,00 %	-45,00 %
Autre	Cas non prévu	0,00 %	0,00 %

Somme Salaires brut pour le projet complet	→	Somme Salaires TTC pour le projet complet
12 960,00 €	→	15 681,60 €

Somme Salaires TTC par jour ouvrés	Pourcentage de taxe payé
163,35 €	17,36 %

Poste	Description du rôle	Année d'expérience	Jours ouvrés	Salaires brut par jour ouvrés	Type d'emploi	Salaires TTC par jour ouvrés	Total brut	Total TTC
Chef de projet	Chef de projet + Game designer + Lead développeur + Directeur Créatif S'occupe de la vision du jeu, de début à la fin.	Junior (0 - 3 ans)	96	135,00 €	CDI	163,35 €	12 960,00 €	15 681,60 €
						0,00 €	0,00 €	0,00 €

Zoom sur les types d'emploi :

Type d'emploi	Description	Multiplieur de coût en taxes	Multiplieur de coût de salaire brut (info)
CDI	Avec le profil actuel de l'entreprise (Auto-entreprise), les charges s'élèvent à 21 %, sinon c'est beaucoup plus.	21,00 %	0,00 %
Freelance	Les personnes en Freelance paient eux même leurs charges, faisant augmenter leur prix. Cependant cela veut dire que nous ne payerons pas de charges.	0,00 %	40,00 %
Stage	Si le stagiaire est payé 600€/mois alors il n'y a pas de taxe, SINON 25 % de taxe sont appliqués en moyenne. (Assurance maladie, vieillesse plafonnée, allocations familiales PAS de chômage, pas de retraite complémentaire, pas d'accident du travail)	25,00 %	-67,00 %
Alternance (Apprentissage)	Le multiplicateur de coût change selon l'âge de la personne, plus est elle âgée, plus l'employeur devra se rapprocher du SMIC (Payé : Accident du travail, Retraite complémentaire, Contribution formation).	6,50 %	-45,00 %
Alternance (Contrat pro)	Le multiplicateur de coût change selon l'âge de la personne, plus est elle âgée, plus l'employeur devra se rapprocher du SMIC (Allègement Fillon et exonérations spécifiques mais pas d'exonération totale)	20,00 %	-45,00 %
Autre	Cas non prévu	0,00 %	0,00 %

Budget alloué aux frais généraux :

Type de dépense / abonnement	Description	Multiplicateur de quantité (nombre)	Coût annuel (€)	Coût mensuel (€)	Coût journalier (€)
Assurance RC Pro	Une assurance qui couvre les dégâts que peut causer une entreprise Lien : https://assurance-moi.fr/guide-pro/responsabilite-cvile/definition	1	0,00 €	0,00 €	0,00 €
Assurance autre	Autres type d'assurance	0	0,00 €	0,00 €	0,00 €
Logiciels et licences	Le coût des logiciels et licences utilisés par l'entreprise	1	0,00 €	0,00 €	0,00 €
Cotisation foncière des entreprises (CFE)	Une taxe pour toutes les entreprises, dont le prix varie selon le chiffre d'affaire annuel de l'entreprise lors de la N-2 année (si en 2025 alors regarder le chiffre d'affaire de 2023) Lien : https://entreprise.service-public.fr/vosdroits/F23547	1	0,00 €	0,00 €	0,00 €
Maintenance matériel informatique	Le coût de maintien en fonctionnement du matériel informatique de l'entreprise	1	120,00 €	10,00 €	0,33 €
Banque / Comptable	Le coût de l'agence de comptabilité et/ou banque	0	0,00 €	0,00 €	0,00 €
Loyer	Le loyer des locaux de l'entreprise	0	0,00 €	1 000,00 €	0,00 €

Type de dépense	Description	Multiplicateur de quantité (kWh)	Coût du kWh (€)	Coût journalier, jour ouvré (€)	Coût annuel ouvré (€)
Électricité	La consommation et le coût d'utilisation de l'électricité, pour faire fonctionner les divers appareils d'une entreprise	0,27 kWh	0,20 €	0,38 €	95,26 €

Coût totaux journalier	Coût totaux journalier ouvré	Somme coût totaux journalier ouvré
0,33 €	0,38 €	0,71 €

Coût totaux mensuel	Coût totaux mensuel ouvré	Somme coût totaux mensuel ouvré
10,00 €	7,94 €	17,94 €

Coût totaux annuel	Coût totaux annuel ouvré	Somme coût totaux annuel ouvré
120,00 €	95,26 €	215,26 €

Budget total :

Calcul du budget total du projet					
Salaires		Frais généraux		Nombre de jour total de travail	Marge imprévu
Salaires TTC pour le projet complet	15 681,60 €	Somme frais totaux par jour ouvrés	0,71 €	96	15,00 %
		Frais généraux pour le projet complet	68,29 €		
Budget total du projet SANS marge d'imprévu			Budget total du projet AVEC marge d'imprévu		
15 749,89 €			18 112,37 €		

Calcul du budget du projet pour X jour de travail (jour ouvrés)					
Salaires		Frais généraux		Nombre de jour total de travail (jour ouvrés)	Marge imprévus
Salaires TTC par jour ouvrés	163,35 €	Somme frais totaux par jour ouvrés	0,71 €	21,66	15,00 %
Salaires TTC pour le nombre de jour donné	3 538,16 €	Frais généraux pour le nombre de jour donné	15,41 €		
Budget du projet SANS marge d'imprévus			Budget du projet AVEC marge d'imprévus		
3 553,57 €			4 086,60 €		
Note : Les valeurs donnés ci-dessus peuvent être fausse si jamais un employé de type Freelance fini son contrat durant la période sélectionnée					

Note : Le budget de 18 112,37 € obtenu ici, est hypothétique.

Le projet fait tout pour ne pas avoir à réellement nécessiter de dépenses.

C'est pourquoi aucun outil payant n'a été utilisé.

B. Planning

Grâce au tableur précédemment créé, on se retrouve déjà avec une sorte de planning, cependant il sera plus intéressant de rendre sa lecture et modification plus facile.

C'est pourquoi un planning sous la forme de calendrier a été créé :

Note : Les cases affichées en rouge représentent une journée alloué à la préparation du passage pour l'obtention CDA.

Calendrier annuel												2026
Janvier	Février	Mars	Avril	Mai	Juin	Juillet	Août	Septembre	Octobre	Novembre	Décembre	
01 Férié	01	01	01	01 Férié	01	01	01	01	01	01 Férié	01	
02	02	02	02	02	02	02	02	02	02	02	02	
03	03	03	03	03	03	03	03 Repos	03	03	03	03	
04	04	04	04 FCM	04	04	04	04	04 Drapeau Ludo	04	04	04	
05 Travail DBU	05	05	05 Férié	05 x2	05	05	05	05	05	05	05	
06	06	06	06 Férié	06	06	06	06	06	06	06	06	
07	07	07	07 Vacances	07 Phase 1 → QA2	07	07	07	07	07	07	07	
08	08	08	08	08 Férié	08	08	08	08 Vacance	08	08	08	
09	09	09	09	09	09	09	09 coupin	09	09	09	09	
10	10	10	10	10	10	10 Camping	10	10	10	10	10	
11	11	11	11	11 Pré-production	11	11	11	11	11	11 Férié	11	
12	12	12	12	12 projet CDA	12	12	12	12	12	12	12	
13	13	13	13	13	13	13	13	13	13	13	13	
14	14	14 Phase 1 → QA1	14	14 Férié	14	14 Férié	14 Repos	14 Préparation	14	14	14	
15	15	15	15	15	15	15	15 Repos	15 Férié	15 diapo	15	15	
16	16	16 Travail + conduite	16	16	16	16 Pas de passage	16	16	16	16	16	
17	17	17 Fix problème	17	17	17	17 pour moi,	17	17 Écriture	17	17	17	
18	18	18 détectés	18	18 Création	18	18 décalé	18	18 documents	18	18	18	
19	19	19 lors QA1	19	19 application CDA	19 x2	19	19	19	19	19 Sortie GTA VI	19	
20	20	20	20	20	20	20	20	20	20	20	20	
21	21	21	21 x2	21	21	21	21	21 Passage CDA	21	21	21	
22	22	22	22 x2	22	22	22 Poursuite	22	22	22	22	22	
23	23	23	23 x2	23	23 développement	23	23	23	23	23	23	
24	24	24	24	24 Férié	24 application CDA	24	24	24	24	24	24	
25	25	25	25	25 Férié	25	25	25	25	25	25	25 Férié	
26	26	26	26	26	26	26 Mami	26	26	26	26	26	
27	27	27	27 x2	27	27	27	27 Mariage Mari...	27	27	27	27	
28	28	28	28	28	28	28	28	28	28	28	28	
29	29	29	29 x2	29	29	29	29	29	29	29	29	
30	30	30	30	30	30	30	30	30	30	30	30	
31		31		31		31	31		31		31	

6. Organisation de développement

Le développement était organisé autour de fonctionnalités et de tâches individuelles.

Lorsqu'une nouvelle fonctionnalité devait être développée, celle-ci était d'abord isolée du reste du projet afin de limiter les risques de régression pendant son développement.

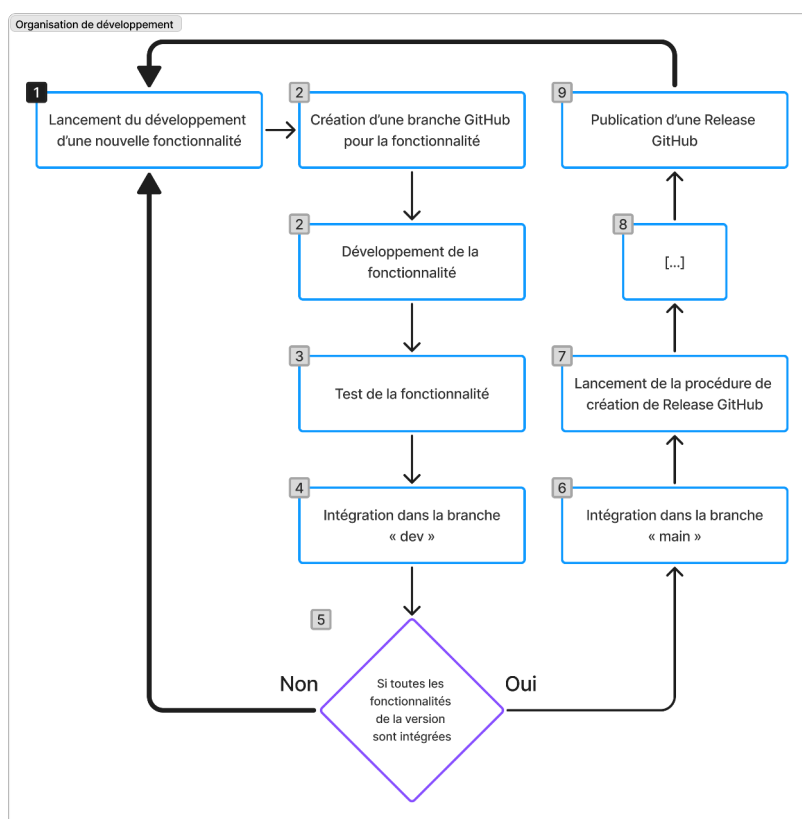
Pour cela, une copie du projet était alors créée via la création d'une nouvelle branche GitHub.

Note : Afin de plus facilement identifier ce type de branches, les branches dédiées au développement de features possèdent le préfixe « feat-[...] ».

Ces copies permettaient de développer et de tester une modification avant son intégration dans la branche de développement (nommée « dev »).

La branche « dev » constituait ainsi l'environnement principal d'intégration des nouvelles fonctionnalités, tandis que la branche « main » représentait la version stable du projet utilisée pour produire les versions officielles (Release).

On se retrouve donc avec deux boucles itératives :



Note : La tâche « [...] » représente les tâches effectuées lors d'une procédure de création de Release GitHub. Ces tâches seront développées dans la section « [Déploiement et DevOps](#) ».

7. Suivi quotidien et Daily

Le suivi du développement était réalisé principalement à l'aide d'un document de suivi quotidien, nommé Daily.

Chaque jour, j'y indiquais principalement :

- les travaux réalisés
- les problèmes rencontrés, ainsi que leurs conséquences
- les tâches à effectuer à court terme
- les éléments nécessitant éventuellement une investigation supplémentaire

Le Daily ne constituait donc pas uniquement une liste de tâches.

Il permettait également de conserver une trace de l'évolution du projet et des problèmes rencontrés pendant son développement.

L'utilisation quotidienne du Daily permettait notamment de vérifier régulièrement l'avancement du projet et d'identifier les éléments susceptibles de prendre plus de temps que prévu.

Cette organisation était volontairement légère.

Dans le contexte d'un projet individuel, je ne considérais pas nécessaire de mettre en place un système de suivi plus lourd (Trello / ClickUp, Gantt).

L'objectif était avant tout de disposer d'une vision suffisamment claire de l'état du projet pour pouvoir prendre les décisions nécessaires au développement afin de respecter les objectifs fixés.

8. Gestion des priorités

Les priorités assignées aux différentes tâches étaient adaptées au fur et à mesure de l'avancement du projet.

Les fonctionnalités indispensables au fonctionnement de l'application étaient traitées en priorité.

Les fonctionnalités secondaires ou pouvant être ajoutées ultérieurement étaient volontairement repoussées lorsque leur développement risquait de compromettre l'avancement des éléments essentiels.

Cette organisation permettait notamment de conserver une version fonctionnelle et présentable du projet même lorsque certaines fonctionnalités n'étaient pas encore développées.

Le développement a ainsi été originellement organisé autour de plusieurs Phases successives, avec notamment la réalisation d'un MVP (Minimum Viable Product).

- **Phase1** : Production du MVP
- **Phase2** : Ajout de plus de feedback / finitions au projet
- **Phase3** : Ajout de features supplémentaires

Difficulté rencontrée :

Cependant, il s'est avéré qu'après la production du MVP (Phase 1), mon système de Phase était prévu uniquement pour des grosses modifications entre les Phases.

Cependant les travaux qui allaient suivre n'étaient pas équivalents aux travaux fournis pour produire la Phase 1.

Par conséquent j'ai choisi de changer pour un système de versionnage comme on peut retrouver de manière générale dans les applications.

Cela me permet de faire des Releases même pour des plus petits changements.

On passe donc à :

- **1.0.0** : Production du MVP
- **1.1.0** : Ajout d'une version du jeu en Anglais
- **1.2.0** : Ajout de plus de feedback / finitions au projet
- **1.2.1** : Ajout de plus de systèmes de sécurité pour le serveur

Note : Chacune de ces versions a une Release associée, vous pouvez donc retrouver le patch-note complet sur le dépôt GitHub du projet : <https://github.com/Alexandre94fr/GaucheOuDroite/releases>

Le MVP avait pour objectif de disposer rapidement d'une version minimale mais fonctionnelle de l'application. Il constituait une base à partir de laquelle les fonctionnalités supplémentaires pouvaient ensuite être ajoutées.

Les versions suivantes permettaient progressivement d'enrichir cette version avec les fonctionnalités complémentaires, des systèmes de sécurité, l'amélioration de l'expérience utilisateur ainsi que les éléments nécessaires à la finalisation du projet.

Note : J'avais décidé que ce MVP, devait aussi contenir les graphismes finaux du jeu.

J'ai pris ce choix, car les graphismes du jeu sont volontairement assez simple à produire, et ne vont donc que très peu ralentir la production du MVP.

L'objectif était qu'en cas de retard grave qui me force à présenter le projet non terminé, je puisse tout de même présenter un projet qui ressemble aux mockups du jeu.

Cette organisation par versions permettait également de réduire le risque de ne pas disposer d'une application fonctionnelle et présentable à l'approche de la date de rendu.

9. Gestion du versionnement GitHub

Le projet a été versionné avec Git, en utilisant GitHub comme dépôt distant et GitHub Desktop comme interface de gestion du dépôt.

Le versionnement avait plusieurs objectifs :

- séparer les développements en cours des versions stables
- pouvoir revenir à un état précédent du projet en cas de problème
- conserver l'historique des modifications
- conserver une trace de l'évolution du projet

Les branches principales utilisées étaient :

- « main » : branche correspondant à l'état le plus stable du projet, chaque version de la branche est associée à une version et à une Release correspondante.
- « dev » : branche utilisée pour intégrer les nouvelles fonctionnalités et modifications avant leur passage dans la version la plus stable

Les modifications étaient enregistrées régulièrement sous forme de commits GitHub afin de conserver un historique suffisamment détaillé du développement.

Le dépôt GitHub constituait ainsi à la fois un outil de versionnement et un élément permettant de suivre l'évolution du projet.

10. Gestion des versions et Releases

Lorsqu'une version du projet était considérée comme suffisamment stable pour être distribuée, une nouvelle version était créée et associée à la branche « main ».

Le projet utilise une convention de versionnement inspirée du Semantic Versioning, sous la forme MAJOR.MINOR.PATCH.

Une version se présente sous la forme de trois nombre séparés par des points, exemple : 1.4.27

Chaque nombre a une signification particulière, plus les modifications faites au projet sont importantes, plus le nombre de la version qui sera modifiée sera proche de la gauche :

- **X.0.0** : Modifications extrêmement importantes du projet (changement d'objectif projet)
- **0.Y.0** : Modifications importantes du projet (ajout de grosses mécaniques)
- **0.0.Z** : Modifications légères du projet (correction de bugs, modification de valeur, ajout de petites mécaniques)

Les versions officielles du projet étaient distribuées sous la forme de Releases GitHub.

Une Release correspondait à une version considérée comme suffisamment stable pour être utilisée et testée indépendamment de l'état actuel du développement.

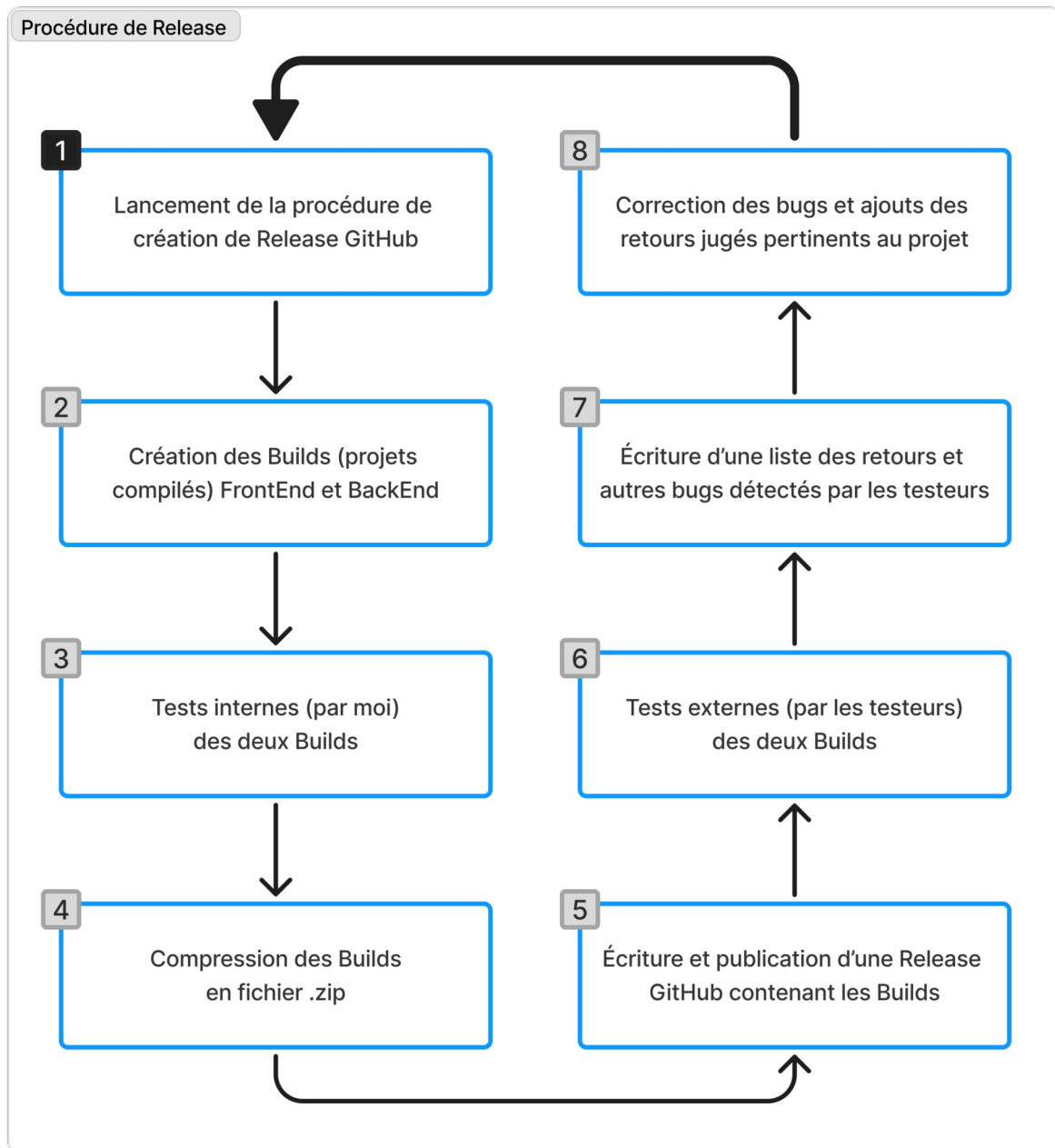
Avant de créer une nouvelle Release, une Build complète du projet était générée à partir de la branche la plus stable, « main ».

Cette Build était ensuite testée afin de vérifier que le fonctionnement général du projet était correct et qu'aucun problème spécifique à la version compilée n'avait été introduit.

Une fois la Build validée, elle était regroupée dans une archive .zip puis associée à une Release GitHub numérotée (le numéro correspond à la version).

La version distribuée pouvait ensuite être testée par des personnes extérieures au projet. Les éventuels bugs et retours utilisateurs étaient listés afin de déterminer les corrections ou améliorations à apporter.

Le processus pouvait alors être répété tout le long de la production du projet :



Cette procédure sera davantage détaillée tout au long de la section « [Déploiement et DevOps](#) », notamment concernant la préparation des Builds et le processus de mise à disposition des versions

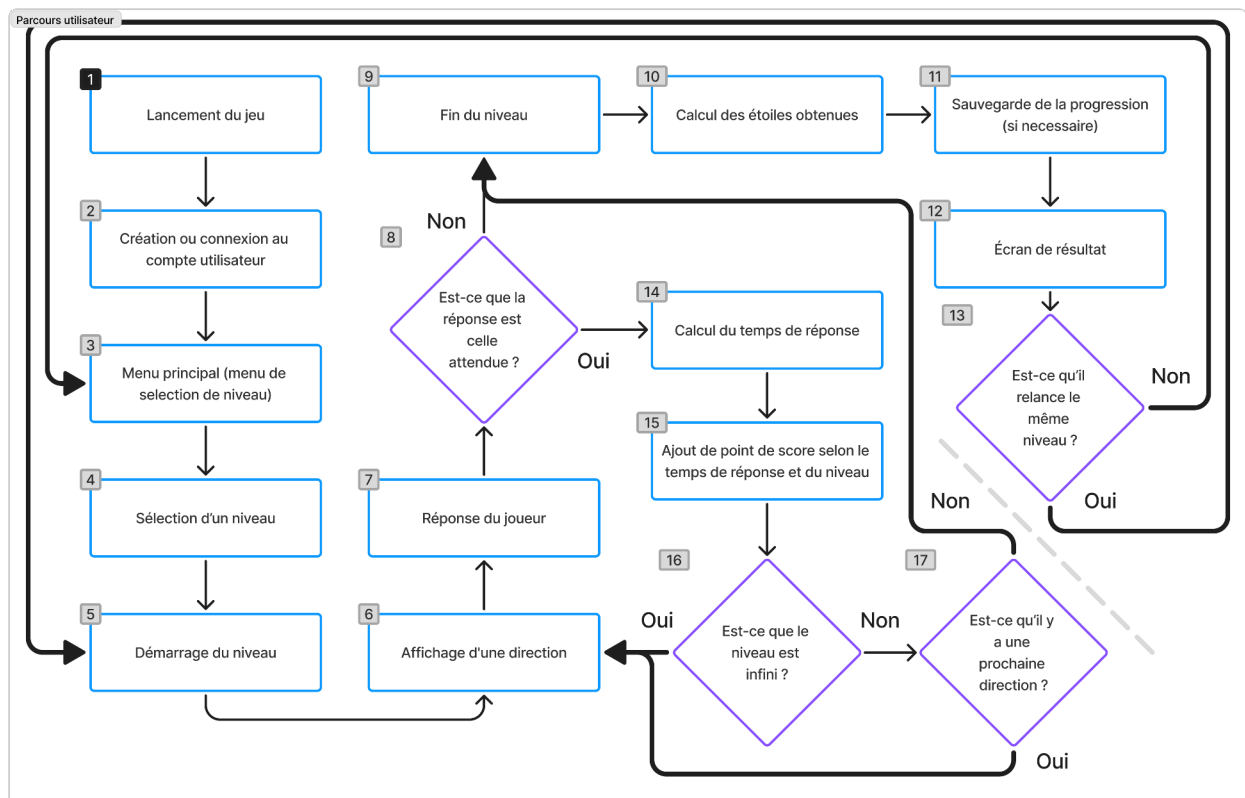
III. Conception et maquettage

1. Parcours utilisateur

Afin d'identifier, de planifier et de valider les différentes étapes du parcours utilisateur, une vue d'ensemble du fonctionnement du jeu a été réalisée.

Cette représentation permet notamment d'anticiper les fonctionnalités nécessaires au projet ainsi que les différents cas rencontrés par l'utilisateur au cours d'une partie.

Une vue d'ensemble du parcours utilisateur a été imaginée :



2. Maquettes et conception des interfaces

Lors de la conception des interfaces, il était possible de commencer par la réalisation de wireframes basse fidélité afin de définir uniquement la structure et l'organisation des différents éléments graphiques, avant de passer à la réalisation des maquettes finales.

Cependant, ce choix n'a pas été retenu pour ce projet.

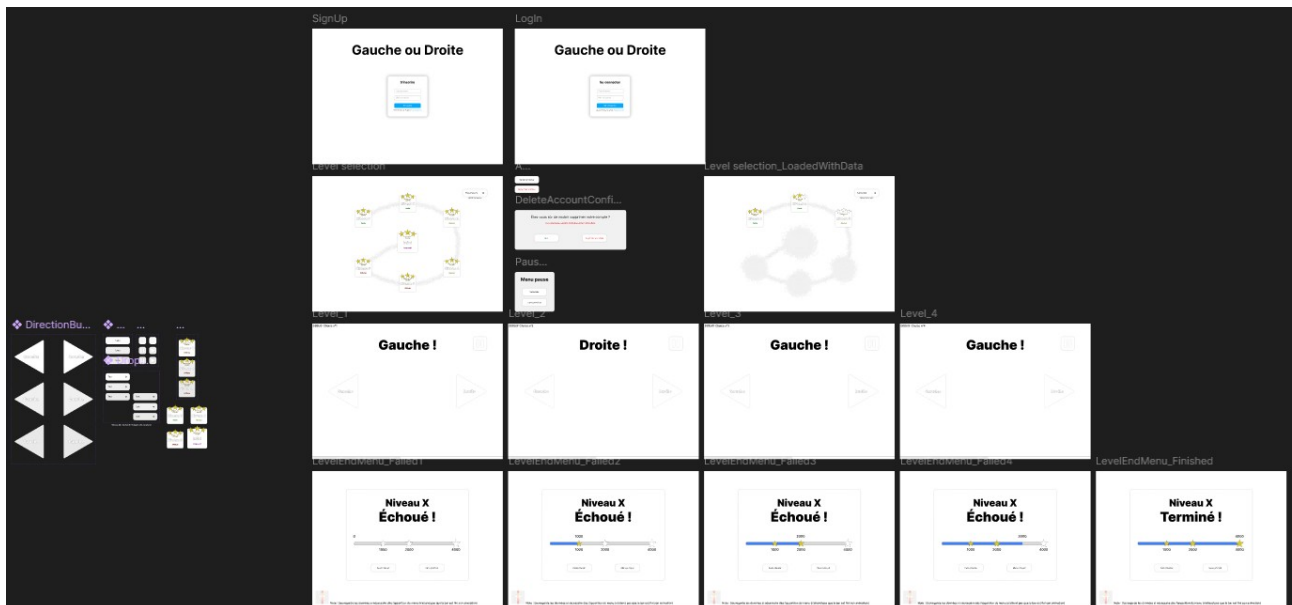
Compte tenu de la simplicité volontaire des interfaces et du temps disponible pour le projet, la réalisation de wireframes en plus d'autres schémas, aurait représenté une étape supplémentaire apportant relativement peu de valeur au processus de conception.

J'ai donc choisi de passer directement à la réalisation des maquettes avec les graphismes définitifs du projet.

Cette approche présentait également l'avantage de pouvoir vérifier directement plusieurs aspects de la conception graphique, notamment la lisibilité des interfaces et la cohérence de la direction artistique choisie.

La direction artistique repose principalement sur une interface sobre, utilisant notamment des éléments blancs accompagnés d'ombres afin de distinguer les différents éléments de l'interface. La réalisation directe des maquettes finales m'a ainsi permis de vérifier que cette approche restait suffisamment lisible et adaptée aux différentes interfaces de l'application.

Les maquettes ont été réalisées avec Figma et ont servi de référence lors de la réalisation des interfaces dans Unity.



3. Interfaces principales

L'une des principales mécaniques du projet est la gestion de l'authentification.

On a donc un écran pour créer son compte et un écran pour se connecter à son compte.

Un utilisateur peu très rapidement passer de l'un à l'autre selon ses besoins.

Gauche ou Droite



S'inscrire

Pseudonyme

Mot de passe

S'inscrire

Vous avez déjà un compte ? [Connectez-vous](#)

Gauche ou Droite



Se connecter

Pseudonyme

Mot de passe

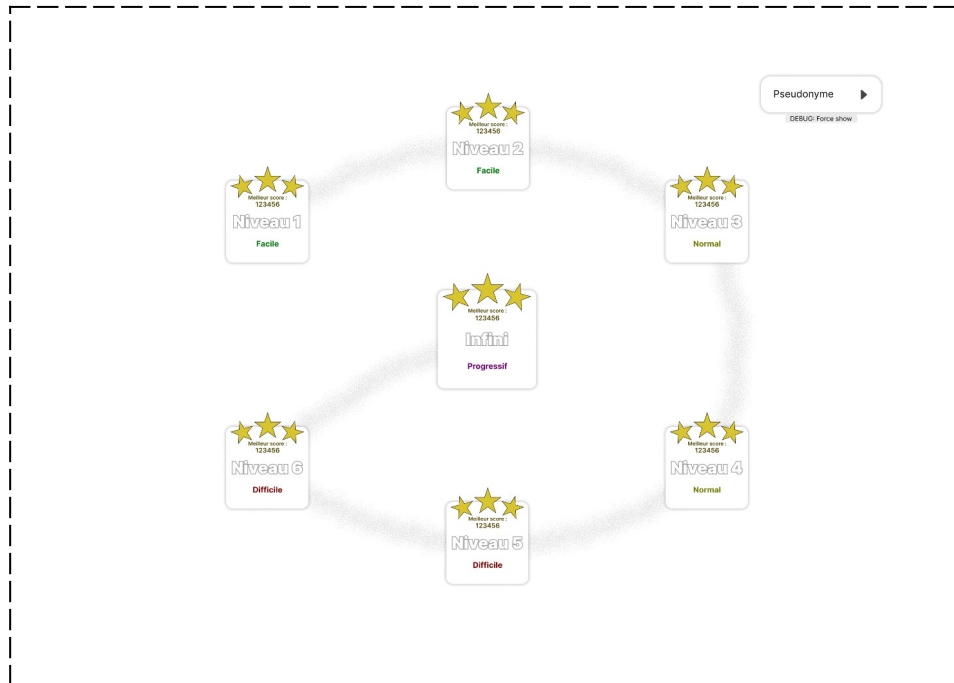
Se connecter

Vous n'avez pas de compte ? [Inscrivez-vous](#)

Une fois authentifié, l'utilisateur accède au menu principal.

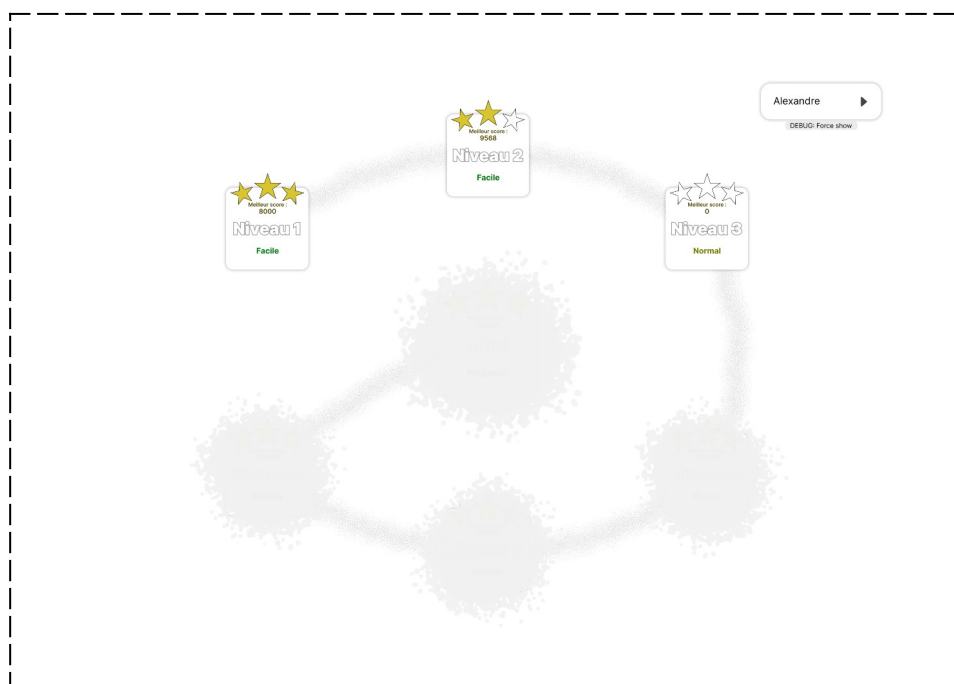
Celui-ci constitue le point central de navigation de l'application.

Il permet notamment de lancer un niveau, mais également d'accéder aux fonctionnalités liées au compte utilisateur, telles que la déconnexion ou la suppression du compte.



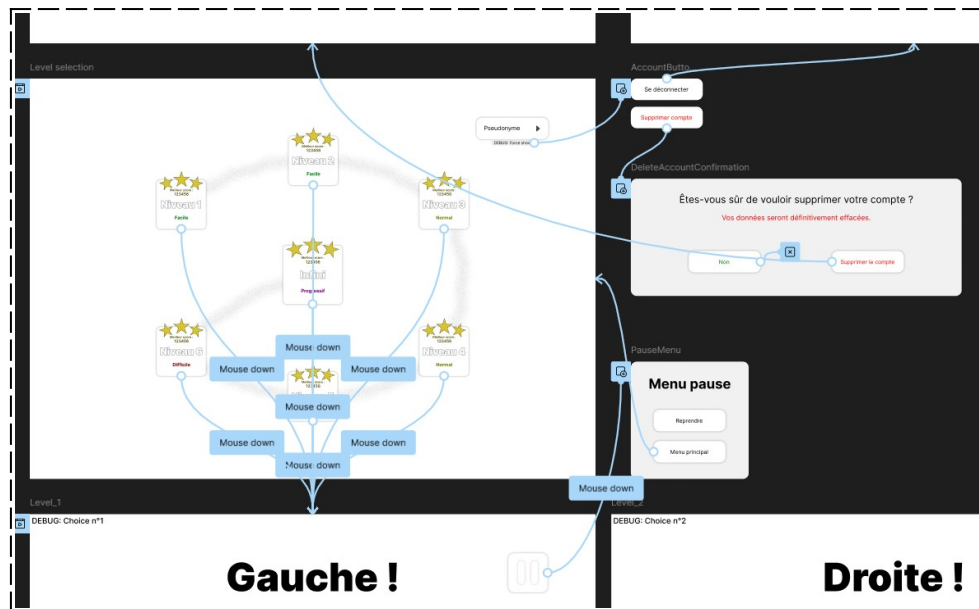
Certaines informations affichées dans les interfaces dépendent des données du jeu et de la progression du joueur.

Ces données sont récupérées après l'authentification depuis la base de données afin d'adapter notamment l'état des niveaux affichés dans le menu principal.



Pour ce projet j'ai décidé d'utiliser une fonctionnalité de Figma, le prototypage. Cela permet d'associer à des éléments graphiques des interactions et une conséquence pour chaque interaction.

Cela m'a permis de simuler le parcours utilisateur et de vérifier la cohérence de la navigation avant même l'implémentation des interfaces dans Unity.



Lorsque l'utilisateur lance un niveau, la partie gameplay du jeu commencera. L'utilisateur devra alors indiquer la direction affichée à l'écran.

Cette maquette permet notamment de vérifier la hiérarchie visuelle des informations importantes pendant une partie : direction à identifier, placement des boutons, temps restant et accès à la pause.

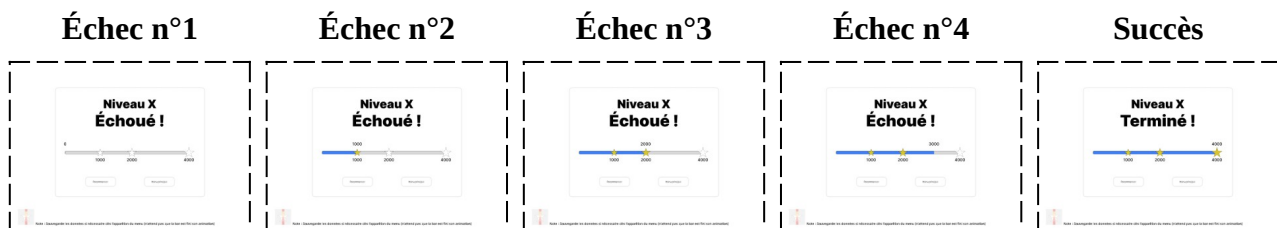


Si le joueur se trompe, ou s'il réussit le niveau, la partie se stoppera et un menu apparaîtra.



L'utilisateur, une fois dans ce menu, peut souhaiter de recommencer le même niveau, ou retourner au menu principal.

Le menu s'adapte au type de fin de partie mais aussi, au score obtenu par le joueur.



4. Choix ergonomiques

Dès la conception du jeu, des raccourcis gameplay ont été planifiés.

Il est par exemple possible d'utiliser les touches Q et D ou les flèches directionnel gauche et droite pour indiquer son choix de direction.

La liste des raccourcis gameplay sont accessibles depuis le menu principal, dès l'authentification de l'utilisateur. Et est affichée à chaque fois que le joueur arrive dans le menu principal, et ce, jusqu'à qu'il batte le premier niveau.

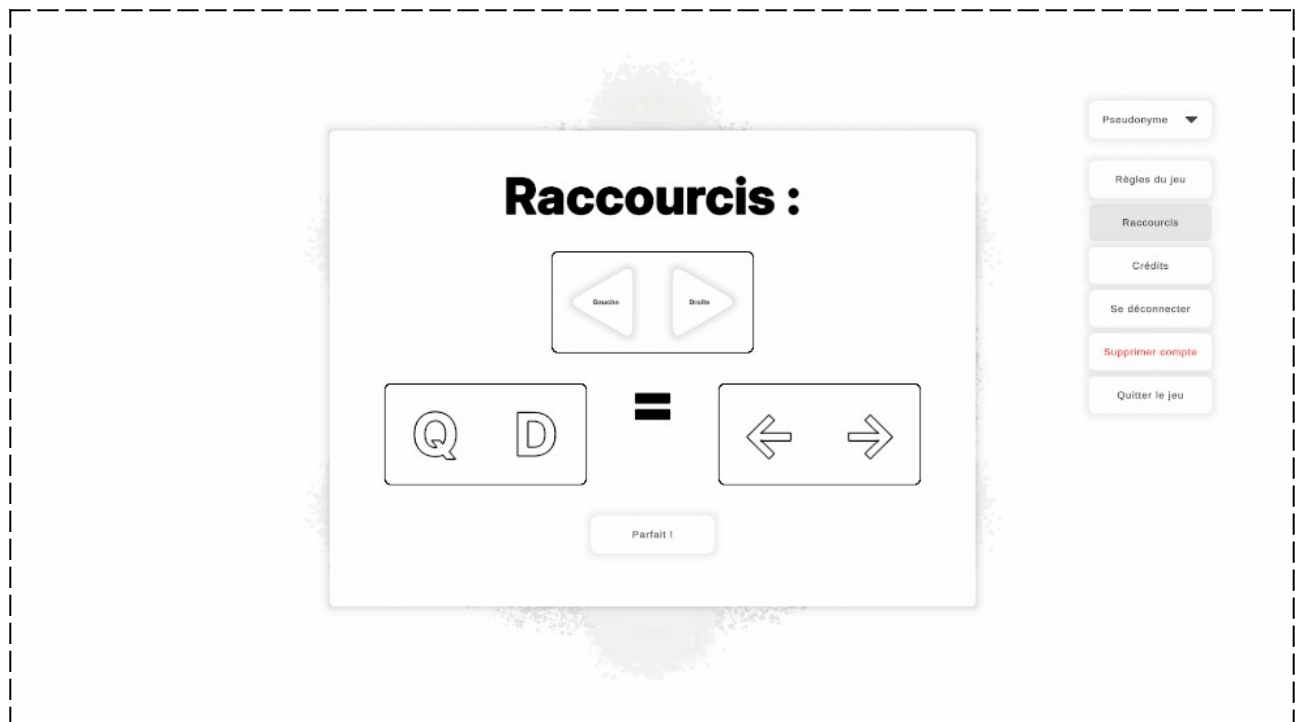


Image provenant du jeu final (depuis Unity, sans chargement des données via base de données)

Plus tard durant le développement du projet, des raccourcis ont été ajoutés aux deux menus d'authentification, dû à des retours de test par les testeurs.

Il est donc possible d'utiliser la touche Tab pour passer d'un élément d'interface à un autre (Pseudonyme → Mot de passe → ...), et d'utiliser les touches Entrée pour lancer la procédure d'authentification.

Il est donc possible de s'authentifier sans nécessiter l'intervention de la souris.

5. Accessibilité et lisibilité

La lisibilité était particulièrement importante pour la partie gameplay, puisque le joueur doit identifier rapidement une information avant de prendre une décision.

Les éléments essentiels ont donc été volontairement limités en nombre et mis en évidence afin de réduire les informations secondaires susceptibles de détourner l'attention du joueur.

La direction artistique repose ainsi principalement sur une interface sobre, utilisant notamment des éléments blancs accompagnés d'ombres afin de distinguer les différents composants de l'interface.

Certains éléments utilisent également des couleurs afin d'améliorer leur contraste et de faciliter l'identification de certaines informations ou actions. Ces couleurs permettent également d'apporter davantage de variété visuelle à l'interface tout en conservant une esthétique générale simple.

Les interfaces adaptent en temps réel leurs positions et leurs tailles en fonction de la taille de l'écran d'ordinateur de l'utilisateur.

Le fonctionnement de ce système a pu être vérifié en condition réelle lors d'une de mes sessions de test par des testeurs (personnes extérieures au projet).

L'un de mes testeurs, m'a fait un retour pour me prévenir que les interfaces ne s'adaptaient par correctement à la taille de son écran (3440x1440).

J'ai donc modifié le projet afin que ça ne soit plus le cas.

À ce jour, les interfaces s'adaptent correctement aux différentes tailles d'écran d'ordinateur testées.

IV. Architecture logicielle

1. Choix technologiques

Afin de développer le FrontEnd du projet, le moteur de développement de jeu vidéo Unity a été utilisé.

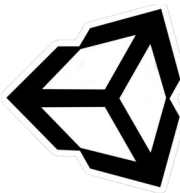
J'avais décidé d'utiliser le moteur Unity car il s'agit d'un moteur que je maîtrise et apprécie utiliser.

Cela m'a permis de gagner du temps sur le développement du jeu lui-même, et me permet donc d'allouer plus de temps à l'apprentissage des compétences que je ne maîtrisais pas encore, et qui étaient nécessaires au bon développement du projet.

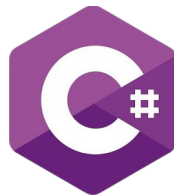
Le langage de programmation associé à Unity est le C#, c'est donc lui qui a été utilisé.

Visual Studio a été utilisé comme IDE (Environnement de Développement Intégré) notamment pour sa bonne intégration avec Unity et son environnement de développement C#.

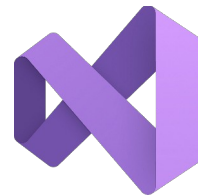
Unity



C#

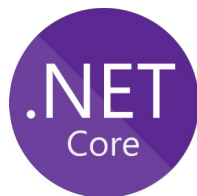


Visual Studio



Afin de développer le BackEnd du projet, les outils et bibliothèques ASP.NET Core 10 Web et Entity Framework Core ont été utilisés.

ASP.NET Core 10



Entity Framework Core



Ces outils et bibliothèques ont été choisis afin de permettre de rendre le C# le langage principal du projet, limitant le nombre de technologies à maîtriser et facilitant les échanges entre le FrontEnd et BackEnd.

Afin de développer la base de données du projet, l'IDE Visual Studio Code, les plugins SQLite et SQLite Viewer, ainsi que le langage de commande SQLite ont été utilisés.

J'avais décidé d'utiliser le langage de commande SQLite car les besoins du projet restent relativement simples et ne nécessitent pas l'utilisation d'un système de gestion de base de données plus complexe.

Visual Studio Code a été choisi comme IDE car deux de ses plugins permettent d'utiliser SQLite dans de bonnes conditions de travail : couleur syntaxique, envoi de retour en cas d'erreur et affichage du contenu d'une base de données en temps réel

Visual Studio Code



SQLite



SQLite Viewer



SQLite



Avec ces choix de technologies, on se retrouve dans une situation avantageuse où l'intégralité du projet, hormis les commandes pour la base de données, utilise le langage de programmation C#.

Toutes ces technologies ont été installées, configurées et reliées entre elles.

Cela m'a permis de disposer d'un environnement de travail adapté, permettant au projet d'être créé, de fonctionner correctement, et d'être maintenu.

Pendant le processus de conception et de développement du projet, du temps a été consacré à la recherche de bonnes pratiques de sécurité et de méthodologies adaptées au développement FrontEnd / BackEnd.

Ces recherches m'ont notamment permis de découvrir et de mettre en œuvre des mécanismes tels que les JWT et le Rate Limiting.

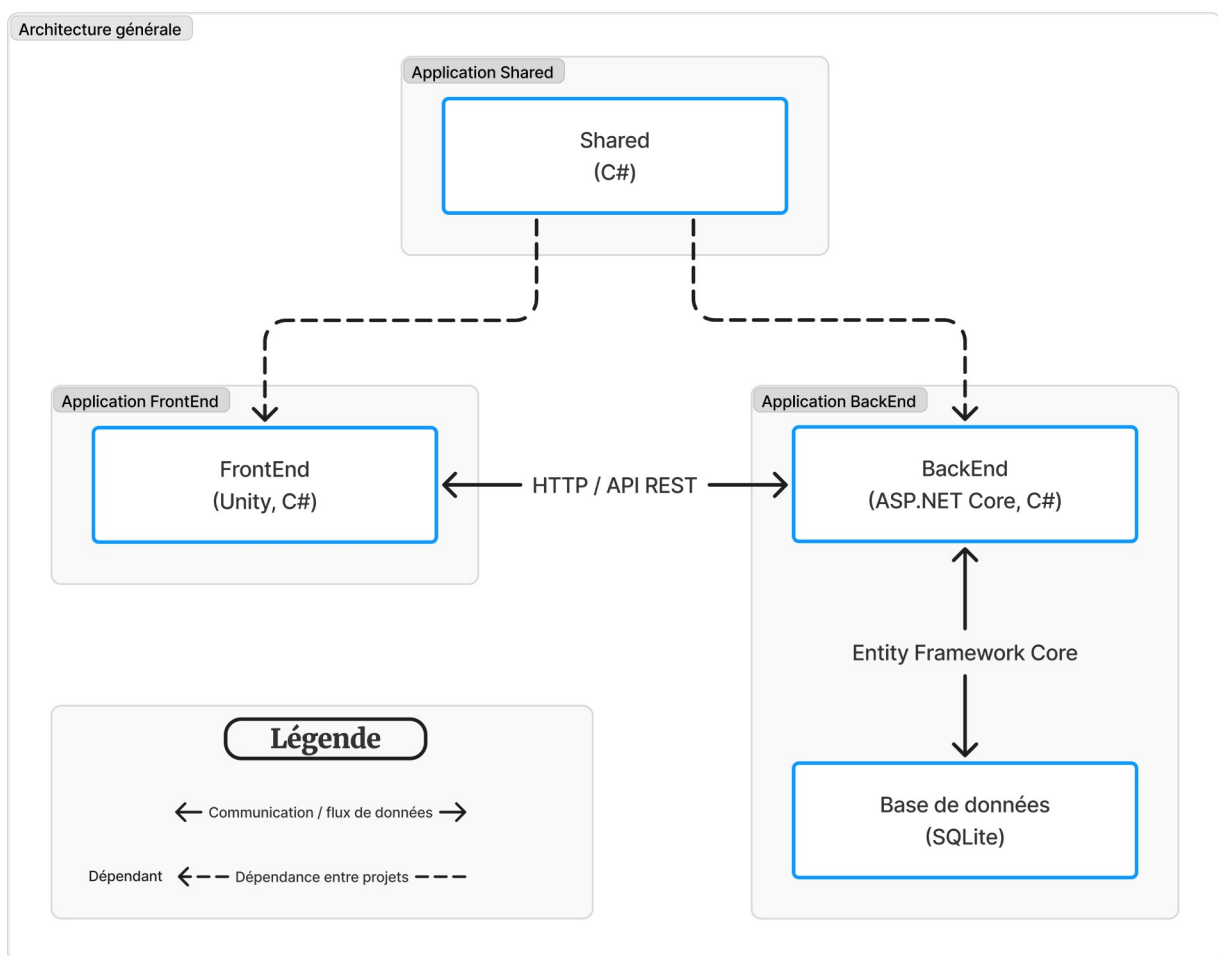
Elles m'ont également permis de comparer différentes solutions et de choisir l'environnement de travail le plus adapté aux contraintes du projet.

2. Architecture générale

Le projet dans sa globalité est organisé en plusieurs applications, chacune avec différents rôles.

- **FrontEnd** : Application Client, c'est ce qui est affiché à l'utilisateur, contient le jeu
- **BackEnd** : Application Serveur, source de vérité, comprend la gestion de la base de données
- **Shared** : Permet de centraliser et de partager aux FrontEnd et BackEnd, les mêmes informations afin qu'ils communiquent efficacement

Les différentes applications peuvent communiquer entre elles si nécessaire.



3. Architecture Client / Serveur

J'ai choisi de séparer mon projet en applications Client / Serveur afin de séparer les responsabilités et de protéger certaines opérations considérées comme sensibles.

Le Serveur / BackEnd est considéré comme la seule source de vérité du projet.

À l'inverse, le Client / FrontEnd est considéré comme une source potentiellement non fiable et ne doit donc jamais être considéré comme digne de confiance par défaut.

Le programme client pouvant être modifié par l'utilisateur, les données importantes et les opérations sensibles doivent toujours être vérifiées par le serveur.

A. Client / FrontEnd

Responsabilités :

- Gestion des Interfaces Utilisateurs (UIs)
- Gestion des interactions utilisateurs (gestion des différents Inputs de l'utilisateur)
- Gestion du Gameplay
- Affichage des différentes données / informations
- Envoi des requêtes à l'API Serveur / BackEnd

B. Serveur / BackEnd

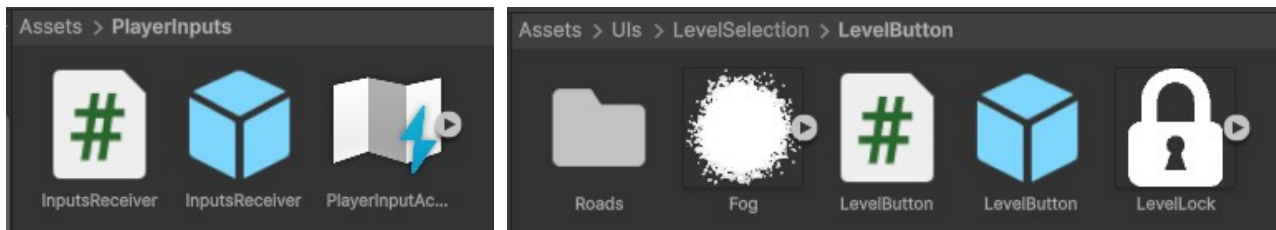
Responsabilités :

- Réception des requêtes du Client / FrontEnd
- Authentification d'utilisateur
- Vérification et validation des données de requête envoyée par le Client / FrontEnd
- Gestion de différents services permettant de répondre aux requêtes reçues
- Gestion de la base de données
- Envoi d'une réponse de requête au Client / FrontEnd

4. Architecture du FrontEnd

J'ai choisi pour le FrontEnd d'avoir une organisation de fichier par fonction et non par type.

Cela veut dire qu'il n'y a pas un dossier Scripts contenant tous les scripts du FrontEnd, mais plutôt un dossier nommé par rapport à la fonction, qui contient le / les scripts de cette fonction, mais aussi les autres assets nécessaires à cette fonction (Prefabs, assets graphiques 2D, etc...)



Cette stratégie permet d'éviter d'avoir à faire des va-et-vient entre différents dossiers éparpillés partout dans le FrontEnd.

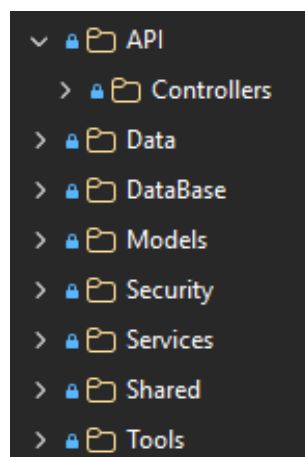
Elle centralise tous les besoins d'une fonctionnalité au même endroit.

5. Architecture du BackEnd

J'ai choisi de faire reposer le BackEnd sur une organisation multicouche permettant de séparer les responsabilités entre la gestion des requêtes HTTP, la logique métier et l'accès aux données.

Cette organisation est notamment matérialisée par la séparation des Controllers, Services et composants liés à l'accès aux données.

D'autres composants, tels que les DTOs, Models, éléments de sécurité et outils utilitaires, viennent compléter cette organisation.



A. Architecture multicouche

Chaque scripts / composants dans un dossier à un rôle spécifique auquel il doit se tenir.
Cela permet de séparer au mieux les responsabilités de chaque composants, rendant le code plus modulaire, plus compréhensible et plus facilement modifiable.

Dossier	Rôle
API.Controllers	Expose les endpoints de l'API au FrontEnd, reçoit les requêtes HTTP du FrontEnd, valide la structure et les données d'entrée de la requête puis appelle les Services correspondant
Services	Contient la principale logique métier du BackEnd
Shared.DTOs	Définit les données échangées entre le FrontEnd et BackEnd
Models	Représente les entités / modèles utilisés par le BackEnd
Data	Contient les fichiers permettant l'accès et la configuration de la base de données
DataBase	Contient les fichiers de commande SQLite ainsi que la base de données elle-même
Security	Contient des scripts utilitaires uniquement liés à la sécurité, tel que le hachage de mot de passe
Tools	Contient des scripts utilitaires, utilisés à plusieurs reprises dans le BackEnd

6. Projet Shared

Le projet Shared permet de répondre à un besoin de centralisation de données / outils communs pour le FrontEnd et BackEnd.

Certaines données comme les DTOs, enums, outils, ou autres constantes ont besoin d'être connues du FrontEnd et du BackEnd.

De plus, il faut que ces données soient les mêmes pour les deux applications.

Par exemple, si on souhaite que le FrontEnd puisse s'assurer qu'il envoie la bonne structure de données au BackEnd, il peut utiliser le DTO correspondant à la requête qu'il envoie au BackEnd.

De son côté, le BackEnd, lorsqu'il recevra la requête de FrontEnd, pourra vérifier que les valeurs reçues sont bien celles attendues (définies par le DTO), et non des données à but malveillant.

C'est pour cela que j'ai créé le projet Shared, elle permet de centraliser ces données et de garantir que le FrontEnd et le BackEnd utilisent les mêmes fichiers partagés.

Ainsi, lorsqu'un élément partagé est modifié, la modification est directement visible par les deux projets et sera prise en compte lors de leur prochaine compilation.

Afin de permettre la communication automatique de Shared aux FrontEnd et BackEnd, j'ai utilisé la commande « mklink » de Windows.

Elle permet de créer un lien, un petit peu comme un pointeur en C++.

Ainsi, ce que voit le FrontEnd et le BackEnd, est le contenu de Shared.

Une légère documentation a été créée afin de permettre de refaire ce processus, si nécessaire :

```
DOCUMENTATION:
=====

If you add new files or folders inside the Shared folder, it will update the Unity version of Shared.

Here is how to create the link:

1.
Start the CDM (console command) in Admin mode
|
2.
mklink /j "C:\Mes jeux\Projet CDA\GaucheOuDroite\ProjectFiles\FrontEnd\GaucheOuDroite\Assets\Shared" "C:\Mes jeux\Projet CDA\GaucheOuDroite\ProjectFiles\Shared"

OR

mklink /d "C:\Mes jeux\Projet CDA\GaucheOuDroite\ProjectFiles\FrontEnd\GaucheOuDroite\Assets\Shared" "C:\Mes jeux\Projet CDA\GaucheOuDroite\ProjectFiles\Shared"

IMPORTANT:
If you want the link to be created, the Shared folder inside the Unity project must not exist when doing the command.

Annexe:
If you want to cancel the link, you need to delete the folders.

Official documentation:
https://learn.microsoft.com/fr-fr/windows-server/administration/windows-commands/mklink
```

Le projet Shared contient les sous-dossiers suivants :

Dossier	Rôle
Constants	Définit les constantes partagées entre le FrontEnd et BackEnd
DTOs	Définit les données échangées entre le FrontEnd et BackEnd
Enums	Définit les énumérateurs partagés entre le FrontEnd et BackEnd
Tools	Définit les outils partagés entre le FrontEnd et BackEnd

7. Communication FrontEnd / BackEnd

Le FrontEnd et le BackEnd communique grâce à des requêtes HTTP.

Les données des requêtes sont transportées dans un JSON.

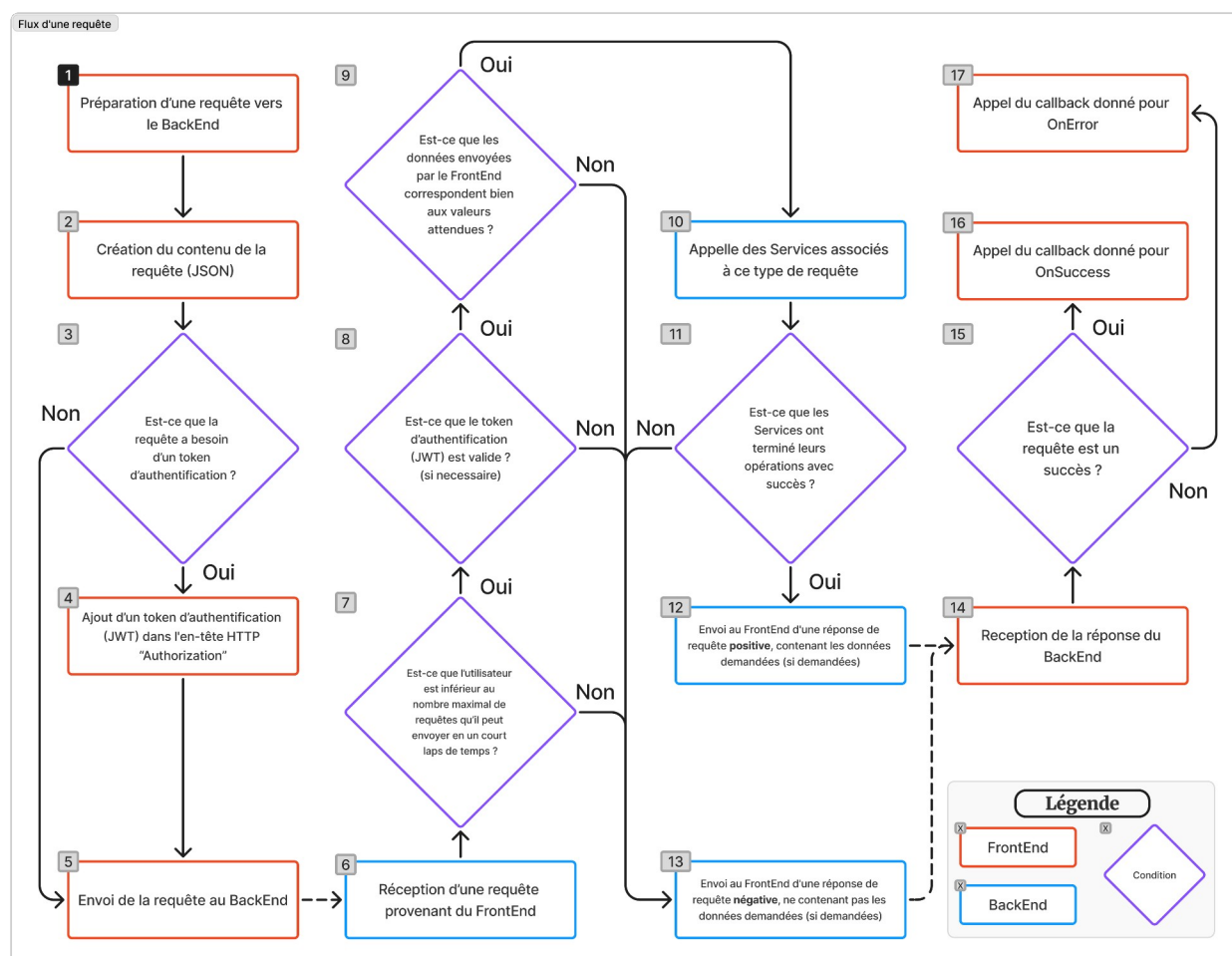
Un token d'authentification est également transmis avec la requête, dans l'en-tête HTTP

« Authorization », s'il ne s'agit pas d'une requête pour s'identifier.

Ce token, généré précédemment par le BackEnd lors de l'authentification, permet à ce dernier de savoir si il parle avec un utilisateur connecté, et si oui, lequel.

On appelle ce token un JWT (JSON Web Token), et permet au FrontEnd de ne pas avoir à envoyer son pseudonyme et mot de passe à chaque requête.

A. Flux d'une requête



V. Conception et persistance des données

1. Besoins en données

En plus de la nécessité de posséder une base de données pour répondre aux demandes du CDA.

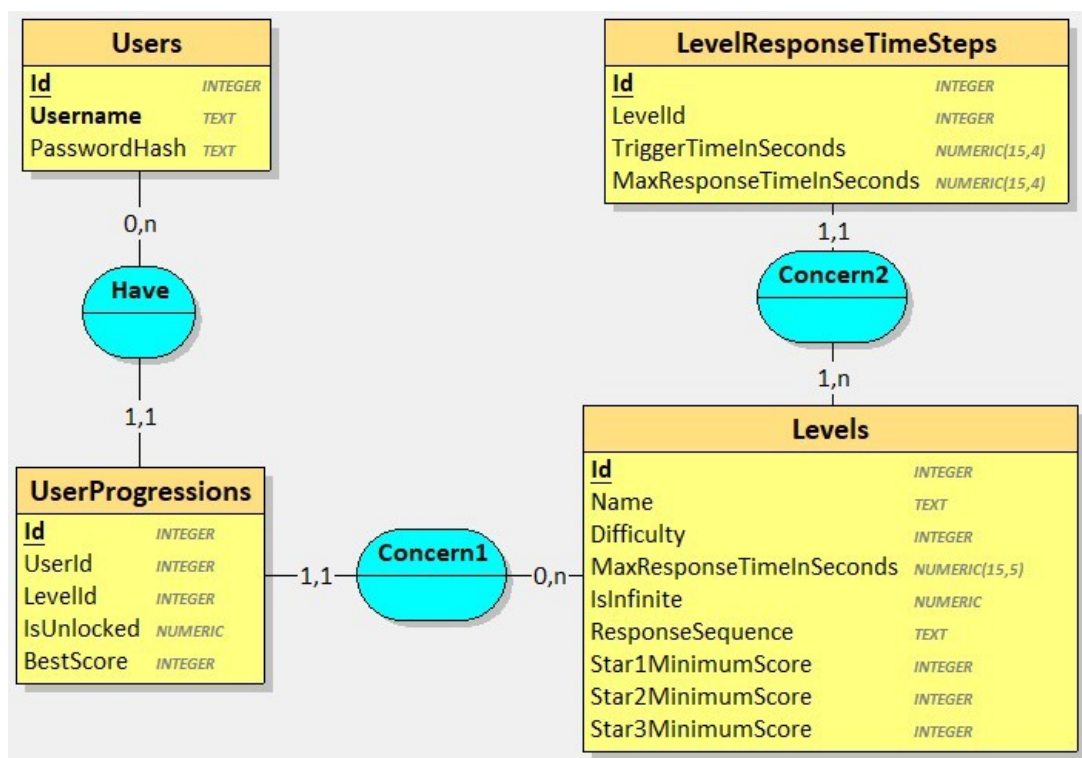
Le choix d'avoir une base de données pour ce projet suit le raisonnement suivant :

- On souhaite sauvegarder les données associées à un joueur entre plusieurs sessions de jeu
- On souhaite pouvoir mettre à jour certaines données du jeu (niveaux) sans avoir à mettre à jour l'application Client de chaque utilisateur

2. MCD

Un schéma MCD (Modèle Conceptuel des Données) représente l'organisation du contenu d'une base de données.

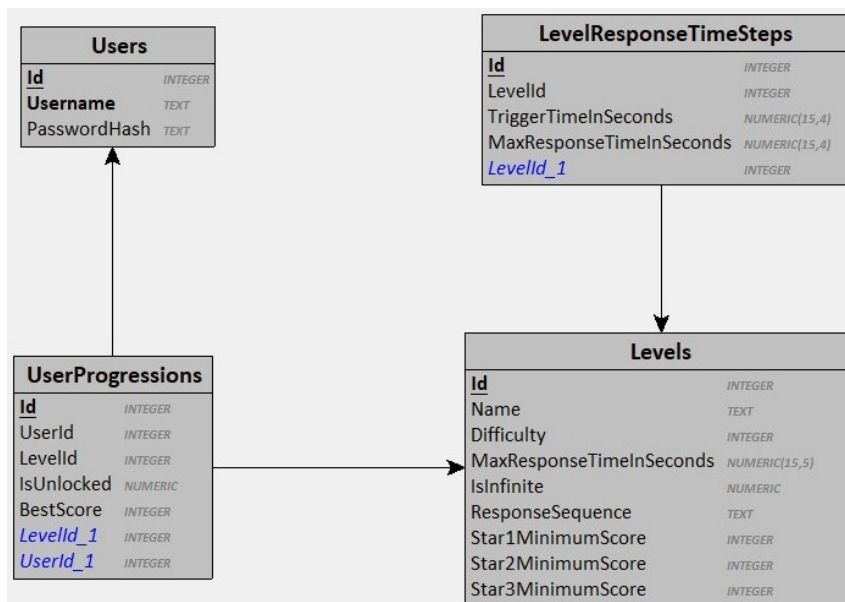
Plus précisément des données nécessaire au projet, de l'organisation des relations entre les différentes données, et ce, indépendamment de leur implémentation technique.



3. MLD

Un schéma MLD (Modèle Logique des Données) traduit le MCD en une structure plus proche de l'implémentation réelle dans une base de données relationnelle.

Il est donc possible dans ce schéma de voir les clés étrangères, ainsi que les sens des dépendances des relations entre les différentes tables.



4. Modèle physique

Le modèle physique correspond à l'implémentation du modèle relationnel dans SQLite.

Les tables, types de données, clés et contraintes sont alors définis conformément aux besoins du projet, et aux schémas précédemment créés.

Pour cela un script de commande SQLite a été créé.

Il permet de créer et d'initialiser la base de donnée.

Ce script n'est exécuté manuellement par un développeur, qu'exclusivement lors de la création ou de la réinitialisation de la base de données. Il n'est donc pas nécessaire de l'exécuter à chaque lancement de l'application.

Lors de son déploiement, le BackEnd dispose déjà d'une base de données créée et initialisée.

Afin d'économiser des pages, les images du code des scripts [Commands.sql](#) et [DataBase.sqlite](#) seront dans l'annexe, à « [1. Script Commands.sql](#) » et « [2. Script DataBase.sqlite](#) ».

Des choix d'optimisation ont été faits afin de minimiser le nombre et la taille des données à sauvegarder :

- Uniquement les meilleurs scores des joueurs sont sauvegardés dans la base de données. Le nombre d'étoiles n'est pas stocké, car il peut être recalculé à partir du score et des seuils définis pour le niveau.
- La propriété « ResponseSequence » dans la table « Levels » a été défini comme une chaîne de caractère où chaque lettre représentent la correcte direction à donner. L signifie Left (Gauche) et R signifie Right (Droite).

Ainsi « ResponseSequence » permet de connaître le nombre de direction à donner pour finir le niveau (sauf si le niveau est un niveau infini), ainsi que chaque direction affichée à l'écran.

Le tout en minimisant la place que ses informations prennent sur la base de données.

Ces choix d'optimisation participent également à l'éco-conception du projet, en limitant notamment la quantité de données nécessaires à sauvegarder et à transmettre lors de certaines opérations.

5. Entity Framework Core et DataBaseContext

Entity Framework Core est l'ORM (Object-Relational Mapping) utilisé par le BackEnd pour permettre à l'application C# de manipuler la base de données à travers des objets et des requêtes plutôt que d'écrire directement toutes les commandes SQL.

Pour permettre la modification de la base de données, on définit un DataBaseContext.

Il constitue le point d'accès d'Entity Framework Core à la base de données. Il définit notamment les ensembles d'entités manipulés par l'application ainsi que la configuration nécessaire à leur correspondance avec les tables de la base de données.

Code provenant du script DataBaseContext

```
1  using Microsoft.EntityFrameworkCore;
2
3  namespace GaucheOuDroiteBackEnd.Data
4  {
5      16 références
6      public class DataBaseContext(DbContextOptions<DataBaseContext> p_options) : DbContext(p_options)
7      {
8          6 références
9          public DbSet<Models.User> Users { get; set; } = default!;
10
11          9 références
12          public DbSet<Models.UserProgression> UserProgressions { get; set; } = default!;
13
14          5 références
15          public DbSet<Models.Level> Levels { get; set; } = default!;
16
17          3 références
18          public DbSet<Models.LevelResponseTimeStep> LevelResponseTimeSteps { get; set; } = default!;
19      }
20 }
```

Afin de par exemple obtenir une valeur dans la base de données, on passe par un objet `DbContext` en utilisant de Linq pour obtenir l'objet que l'on souhaite.

```
103 3 références
104 public async Task<User?> GetUserAsync(string p_username)
105 {
106     if (IS_DEBUG_MODE_ON)
107         Console.WriteLine($"DEBUG: [{GetType().Name}] Starting to try getting the User (Username: {p_username}) from the DataBase.");
108
109     User? user = await _dbContext.Users.FirstOrDefaultAsync(user => user.Username == p_username);
110
111     if (user == null)
112     {
113         if (IS_DEBUG_MODE_ON)
114             Console.WriteLine($"DEBUG: [{GetType().Name}] Failed to find the User (Username: {p_username}) inside the DataBase. Returning null.");
115         return null;
116     }
117
118     if (IS_DEBUG_MODE_ON)
119         Console.WriteLine($"DEBUG: [{GetType().Name}] Successfully got the User (Username: {p_username}). Returning the User.");
120
121     return user;
122 }
```

```
124 3 références
125 public async Task<User?> GetUserAsync(int p_id)
126 {
127     if (IS_DEBUG_MODE_ON)
128         Console.WriteLine($"DEBUG: [{GetType().Name}] Starting to try getting the User (Id: {p_id}) from the DataBase.");
129
130     User? user = await _dbContext.Users.FindAsync(p_id);
131
132     if (user == null)
133     {
134         if (IS_DEBUG_MODE_ON)
135             Console.WriteLine($"DEBUG: [{GetType().Name}] Failed to find the User (Id: {p_id}) inside the DataBase. Returning null.");
136         return null;
137     }
138
139     if (IS_DEBUG_MODE_ON)
140         Console.WriteLine($"DEBUG: [{GetType().Name}] Successfully got the User (Id: {p_id}). Returning the User.");
141
142     return user;
143 }
```

Code provenant des deux méthodes `GetUserAsync()` du script `UserService`

Dans le cas où l'on souhaite ajouter une donnée dans la base de données, on passe encore une fois par un objet `DbContext`, mais cette fois on utilise les méthodes :

- `Add()`, pour ajouter une nouvelle entité au contexte Entity Framework Core
- `SaveChangesAsync()`, pour appliquer les modifications suivies par le contexte à la base de données

```
105 UserProgression userProgression = new()
106 {
107     // The Id will be generated automatically by the DataBase
108     UserId = p_userId,
109     LevelId = p_levelId,
110
111     IsUnlocked = false,
112     BestScore = 0
113 };
114
115 _dbContext.UserProgressions.Add(userProgression);
116
117 await _dbContext.SaveChangesAsync();
```

Code provenant de la méthode `CreateUserProgressionAsync()` du script `UserProgressionService`

6. Persistance des données

Grâce à l'implémentation d'une base de données pour notre projet, on a pu répondre à nos besoins.

Cela comprend la sauvegarde des données utilisateurs, et donc, de leur persistance à travers les différentes sessions de jeu.

Cependant, toutes les données créées en jeu ne sont pas sauvegardées dans la base de données.

Toutes les données manipulées par l'application n'ont pas vocation à être conservées. Seules les données nécessaires entre plusieurs sessions de jeu sont conservées dans la base de données.

Les autres données, uniquement nécessaires au déroulement d'une partie, restent temporaires et sont conservées côté Client.

Donnée	Est-elle persistante ?
Compte utilisateur (pseudonyme + mot de passe haché)	Oui
Meilleur score pour chaque niveau	Oui
Score actuel d'une partie	Non
Nombre d'étoiles obtenues durant une partie	Non

7. Sauvegarde / restauration

Étant donné que la base de données du projet est stockée sous la forme d'un fichier SQLite. Une sauvegarde (backup) peut être réalisée en conservant une copie de ce fichier.

Ainsi, en cas de perte, de corruption ou d'un reset, une copie valide peut être restaurée afin de récupérer les données précédemment sauvegardées, et donc, de limiter la perte de données.

Durant le développement du projet, un système de backup comme décrit plus haut n'a pas été utilisé.

J'ai choisi d'utiliser GitHub pour éviter la modification accidentelle du contenu de la base de données, et donc, la perte de données.

Lorsqu'une modification de la base de données était détectée par GitHub, il suffisait d'annuler cette modification depuis GitHub Desktop.

VI. Développement des composants

1. Organisation du code

Afin de faciliter la maintenance et le débogage du projet, plusieurs principes d'organisation ont été appliqués lors du développement.

Côté FrontEnd, les scripts et ressources sont regroupés par fonctionnalité, comme présenté dans la section « [IV. Architecture logicielle](#) ».

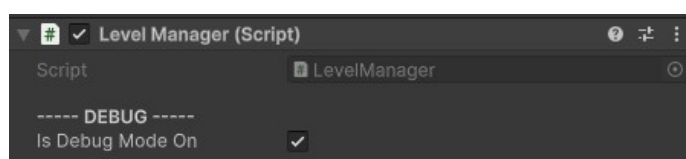
Cette organisation permet de retrouver plus facilement l'ensemble des éléments nécessaires à une fonctionnalité et de limiter la dispersion du code.

J'ai apporté une attention particulière à également facilité le débogage, que ça soit dans le FrontEnd ou BackEnd.

Un système de logs de débogage a été intégré aux différents scripts du projet. Ces messages peuvent être activés ou désactivés individuellement grâce à un paramètre « IsDebugModeOn » présent dans les scripts concernés.

Cette approche permet de conserver les informations utiles au débogage directement dans le code, sans avoir à ajouter puis supprimer des messages de débogage temporaires à chaque recherche de problème. Elle permet aussi de limiter les informations affichées dans la console lorsque le débogage n'est plus nécessaire.

Exemple : Script LevelManager (FrontEnd)



```
383 1 référence
384 bool TryUnlockLocallyNextLevel(out bool p_isNextLevelAlreadyUnlocked)
385 {
386     int nextLevelId = _currentLevelId + 1;
387     p_isNextLevelAlreadyUnlocked = false;
388
389     if (!_userManager.TryGetLevelProgression(nextLevelId, out LevelProgression nextLevelProgression))
390     {
391         if (_isDebugModeOn)
392             Debug.Log($"DEBUG: [{GetType().Name}] Failed to get the {nameof(LevelProgression)} {nextLevelId}. There is no {nameof(LevelProgression)} associated with {nextLevelId}.");
393         return false;
394     }
395
396     if (nextLevelProgression.IsUnlocked)
397     {
398         if (_isDebugModeOn)
399             Debug.Log($"DEBUG: [{GetType().Name}] The {nameof(Level)} {nextLevelId} is already unlocked. Returning false.");
400         p_isNextLevelAlreadyUnlocked = true;
401         return false;
402     }
403
404     nextLevelProgression.IsUnlocked = true;
405     _userManager.UpdateLevelProgression(nextLevelId, nextLevelProgression);
406
407     if (_isDebugModeOn)
408         Debug.Log($"DEBUG: [{GetType().Name}] Successfully saved locally that next {nameof(Level)} (Id: {nextLevelId}) is unlocked. Returning true.");
409     return true;
410 }
```

2. Composants FrontEnd

Les scripts du FrontEnd ont été organisés selon leur responsabilité afin de limiter les dépendances entre les différents scripts. Parmi les différentes catégories existantes, deux importantes peuvent notamment être distinguées :

- les scripts liés au gameplay
- les scripts liés aux interfaces utilisateur

Les scripts gameplay prennent en charge les règles et le fonctionnement du jeu, tandis que les scripts d'interface sont responsables de l'affichage et des interactions avec les éléments de l'UI.

Cette séparation permet d'éviter qu'un script gameplay dépende directement d'un script d'interface particulier.

Une fonctionnalité gameplay peut ainsi être utilisée indépendamment de l'interface qui l'affiche.

Lorsqu'un script gameplay doit provoquer une modification de l'interface, il utilise un système d'événements basé sur des Action de C#. Les événements sont centralisés dans un script dédié afin de permettre aux différents scripts de communiquer entre eux sans créer de dépendance directe entre le gameplay et une interface particulière.

Par exemple, un script lié au gameplay peut déclencher un événement indiquant qu'une nouvelle direction doit être affichée. Le script responsable de l'interface peut alors réagir à cet événement et mettre à jour l'affichage.

Script EventHandler (FrontEnd)

```
7 public class EventHandler
8
9 // DOCUMENTATION:
10
11 // In here we store locally all the Actions (Events) we will need for this project.
12
13 // The reason why we don't have a cleaner Actions handling system,
14 // is because we have little time to make this project and because
15 // we don't have that many Event in our project.
16
17 // -- Input events -- //
18
19 7 références
20 public static Action<DirectionProperties.Direction> OnDirectionChoiceInputEvent;
21
22 6 références
23 public static Action OnPauseInputEvent;
24
25 // Authentication shortcuts
26 3 références
27 public static Action OnAuthenticationNavigationInputEvent;
28 3 références
29 public static Action OnAuthenticationValidationInputEvent;
30
31 // -- Levels events -- //
32
33 5 références
34 public static Action<float> OnMaximumRemainingResponseTimeChangedEvent;
35 3 références
36 public static Action<float> OnRemainingResponseTimeChangedEvent;
37
38 6 références
39 public static Action<DirectionProperties.Direction> OnNextLevelDirectionComputedEvent;
40
41 4 références
42 public static Action<ResponseResult> OnCorrectResponseProcessedEvent;
43 3 références
44 public static Action<int> OnScoreChangedEvent;
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539

```

3. Composants BackEnd

Les scripts du BackEnd ont été développés en suivant l'organisation présentée dans la section « [IV. Architecture logicielle](#) ».

Les DTOs, Controllers et Services permettent notamment de représenter les données échangées, de gérer la réception des requêtes et d'effectuer le traitement des opérations.

Plutôt que de détailler à nouveau le rôle de ces composants, voici quelques extraits du code permettant d'illustrer leur mise en œuvre :

DTO : UserProgressionDTO + LevelProgressionDTO

```
3 namespace Shared.DTOs.Data.User
4 {
5     1 référence
6     public class UserProgressionDTO
7     {
8         16 références
9         public List<LevelProgressionDTO> LevelProgressions { get; set; } = new();
10
11         // Add other properties if necessary
12     }
```

```
1 namespace Shared.DTOs.Data.User
2 {
3     7 références
4     public class LevelProgressionDTO
5     {
6         3 références
7         public int Id { get; set; } = -1;
8
9         // There is no need to store UserId, because the Server can get it directly.
10        // Plus, the Client doesn't need to store this information.
11
12        13 références
13        public int LevelId { get; set; } = -1;
14
15        4 références
16        public bool IsInLocked { get; set; } = false;
17
18        5 références
19        public int BestScore { get; set; } = -1;
20
21        // Add other properties if necessary
22    }
```

Controller : UserProgressionController

```
15 namespace GaucheDuDroitBackend.API.Controllers
16 {
17     [ApiController]
18     [Route("api/user-progressions")]
19     0 références
20     public class UserProgressionController(LevelService p_levelService, UserProgressionService p_userProgressionService, DbContext p_dataBaseContext) : ControllerBase
21     {
22         11 références
23         const bool IS_DEBUG_MODE_ON = true;
24
25         // The DataBase stores the BestScore using an 'int', we don't want a User to break the DataBase by passing a value bigger than an 'int'.
26         1 référence
27         const int INFINITE_LEVEL_MAXIMAL_SCORE = int.MaxValue;
28
29         1 référence
30         readonly LevelService _levelService = p_levelService;
31
32         3 références
33         readonly UserProgressionService _userProgressionService = p_userProgressionService;
34
35         1 référence
36         readonly DbContext _dataBaseContext = p_dataBaseContext;
37
38
39         // Helping methods
40
41         [Authorize]
42         [HttpGet]
43         0 références
44         public async Task<ActionResult> Get()
45         {
46             // ...
47         }
48
49         [Authorize]
50         [HttpPost]
51         0 références
52         public async Task<ActionResult> Put(UserProgressionDTO p_userProgressionDTO)
53         {
54             // ...
55         }
56     }
```

Service : UserProgressionService

```
1 namespace GaucheDuDroitBackend.Services
2 {
3     7 références
4     public class UserProgressionService(DbContext p_dataBaseContext, UserService p_userService)
5     {
6         20 références
7         const bool IS_DEBUG_MODE_ON = true;
8
9         17 références
10        readonly DbContext _dataBaseContext = p_dataBaseContext;
11
12        2 références
13        readonly UserService _userService = p_userService;
14
15
16        // Is existing
17
18        #region - Create -
19
20        /// <summary>
21        /// Creates and saves in the DataBase a new UserProgression. <para></para>
22        /// Checks before creating any UserProgression, if the given UserId and LevelId are not valid <b>AND</b> if a UserProgression with the same UserId and LevelId already exists. <para></para>
23        /// In that case, it returns null and prints out a warning. <para></para>
24        /// <b>Note:</b> If you want to create all the UserProgressions for a newly created User, use the 'see cref=<CreateAllUserProgressionsAsync>' method instead.
25        /// <summary>
26        /// <param name="p_userId"></param>
27        /// <param name="p_levelId"></param>
28        /// <returns> The created UserProgression or null if any check fails. </returns>
29        0 références
30        public async Task<UserProgression> CreateUserProgressionAsync(int p_userId, int p_levelId)
31        {
32            // ...
33        }
34
35        /// <summary> Creates and saves in the DataBase all the UserProgressions a newly...
36        2 références
37        public async Task<List<UserProgression>> CreateAllUserProgressionsAsync(int p_userId)
38        {
39            // ...
40        }
41
42        #endregion
43
44        // Get
45
46        // Update
47
48        // Delete
49    }
```

4. Communication avec l'API BackEnd

La communication entre le FrontEnd et l'API est centralisée dans le script `ServerRequestManager`. Ainsi, les autres scripts du FrontEnd n'ont pas besoin de gérer directement la création des requêtes HTTP.

`ServerRequestManager` prend notamment en charge la préparation des requêtes, leur envoi vers l'API, la réception des réponses et leur conversion en données utilisable (désérialisation) dans les objets attendus.

Cette centralisation permet d'utiliser un fonctionnement commun pour les différentes fonctionnalités nécessitant une communication avec le BackEnd.

Ce script est volontairement plus documenté que les autres afin de me permettre de me rappeler comment il fonctionne, mais aussi de comprendre ses détails techniques.

Afin d'économiser des pages, les images du code du script `ServerRequestManager` seront dans l'annexe, à « [3. Script ServerRequestManager](#) ».

Note : Si jamais vous souhaitez consulter l'intégralité du code du projet, c'est possible en passant par mon dépôt GitHub : <https://github.com/Alexandre94fr/GaucheOuDroite>.

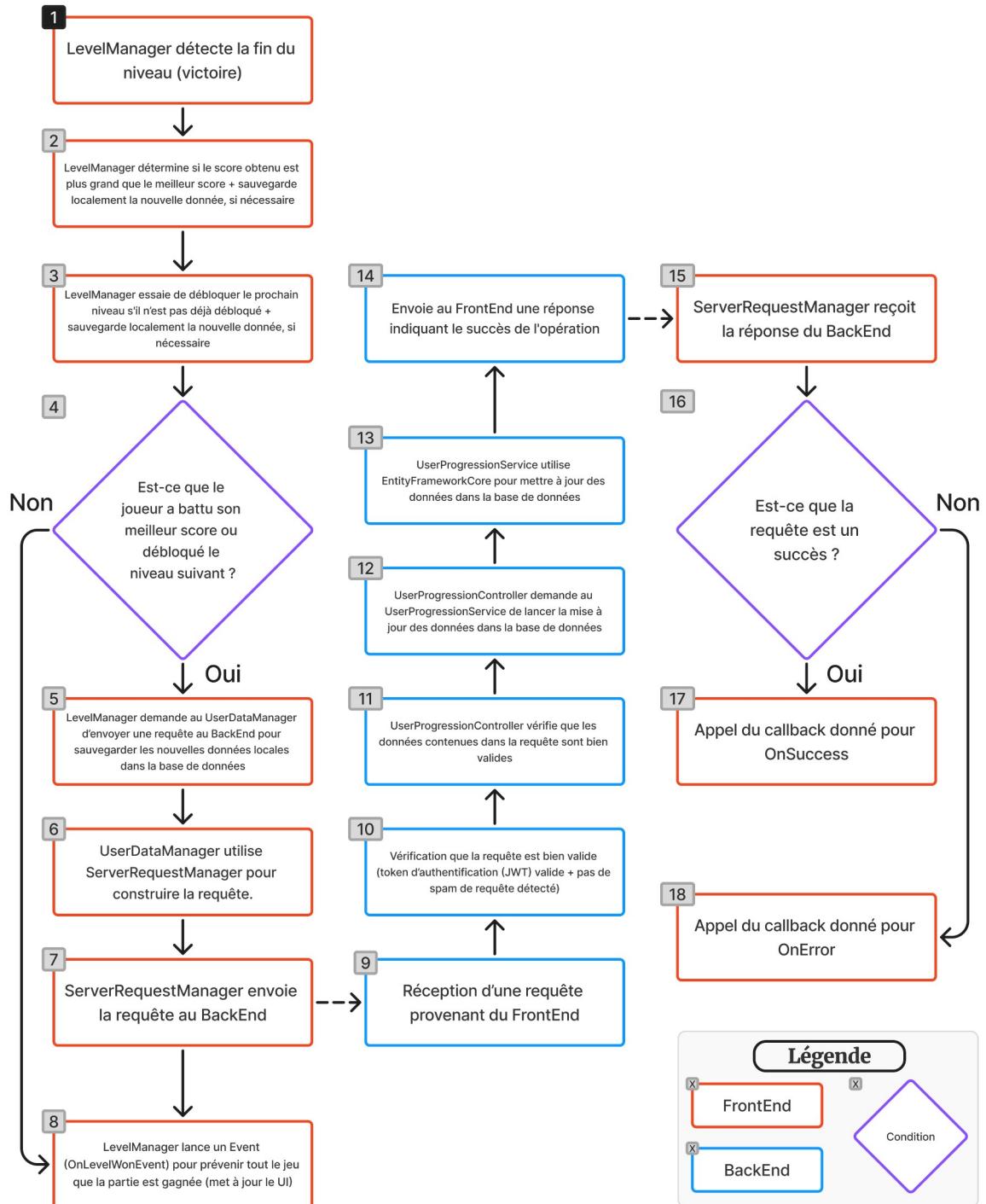
5. Exemple complet d'une fonctionnalité

Le `LevelManager` coordonne les différents scripts nécessaires au déroulement d'une partie.

Les différents scripts coordonnés par le `LevelManager` sont notamment responsables de la gestion du déroulement d'un niveau, du calcul du temps de réponse, de l'évolution du score et de la détermination des conditions de fin de partie.

Afin d'illustrer concrètement l'organisation des scripts présentée précédemment, le flux de la gestion de fin de niveau en cas de victoire du joueur sera utilisé comme exemple :

Flux fin de niveau (victoire)



VII. Sécurité

1. Principes de sécurité

La sécurité a été prise en compte lors de la conception de l'architecture afin de protéger les comptes utilisateurs, les données enregistrées dans la base de données et les fonctionnalités accessibles depuis l'API.

Le principal principe appliqué est de **ne jamais considérer les données provenant du FrontEnd comme fiables**.

Le client Unity peut être modifié ou contourné par un utilisateur. De plus, il est également possible d'envoyer directement des requêtes HTTP à l'API sans avoir à passer par le jeu.

Ainsi, ajouter uniquement des contrôles de sécurité côté FrontEnd ne permettrait pas de sécuriser l'application.

Par conséquent, différents types de contrôles de sécurité ont été implémentés côté BackEnd. Le BackEnd constitue ainsi la frontière de confiance entre le Client et la base de données.

Plusieurs mécanismes de sécurité ont été mis en place dans le projet afin de limiter les principaux risques identifiés :

- protection des mots de passe utilisateur via leur hachage
- protection contre le lancement excessif de requêtes
- authentification de l'utilisateur à l'origine d'une requête via un JWT (JSON Web Token), afin d'éviter l'usurpation d'identité, et l'envoi de données d'identification à chaque requête
- validation incontournable des données reçues lors d'une requête
- limitation des risques d'injection SQL grâce aux validations et à Entity Framework Core

2. Authentification

L'application dispose d'un système d'authentification permettant à chaque utilisateur de disposer d'un compte personnel et de retrouver sa progression entre différentes sessions de jeu.

Deux opérations principales sont proposées :

- la création d'un compte
- la connexion à un compte existant

Lors de la création d'un compte, le FrontEnd transmet les informations nécessaires à l'API. Le BackEnd, plus précisément le script AuthenticationController, vérifie la validité des données reçues avant de poursuivre le traitement de la requête.

Après quoi, le script AuthenticationService fait d'autres vérifications pour, par exemple, vérifier qu'un compte utilisateur avec le même pseudonyme n'existe pas déjà.

Si les vérifications sont réussies, alors le compte sera créé, et le pseudonyme ainsi que le mot de passe donnés par l'utilisateur seront ajoutés à la base de données.

Cependant, le mot de passe n'est jamais enregistré directement en clair.

Il est d'abord transformé en forme haché avant d'être ajouté à la base de données, le rendant volontairement illisible aux yeux d'un humain.

Exactement comme pour la création d'un compte :

Lors d'une connexion, le FrontEnd transmet les informations nécessaires à l'API. Le BackEnd, plus précisément le script AuthenticationController, vérifie la validité des données reçues avant de poursuivre le traitement de la requête.

Après quoi, le script AuthenticationService vérifie si les informations d'authentification (pseudonyme et mot de passe, préalablement haché par le BackEnd) correspondent à un compte existant, si oui, alors l'authentification est considérée comme réussie.

Peu importe que la requête d'authentification ayant réussi soit pour la création ou la connexion à un compte, **le serveur génère et envoie toujours au client un JWT, afin que l'utilisateur puisse s'identifier lors des requêtes suivantes.**

3. Protection des données sensibles

Parmi les données enregistrées, les mots de passe nécessitent une protection particulière, car leur compromission pourrait permettre l'accès aux comptes utilisateurs.

A. Hachage des mots de passe

Pour protéger cette donnée, elle est rendu volontairement illisible aux yeux d'un humain.

Pour cela on utilise un système de hachage.

Le système de hachage du projet est un wrapper du système de hachage d'ASP.NET CORE Identity Framework.

```
3 références
5 public class PasswordHasher
6 {
7     2 références
8     readonly PasswordHasher<object> _hasher = new();
9
10    1 référence
11    public string HashPassword(string p_password)
12    {
13        return _hasher.HashPassword(
14            null!,
15            p_password
16        );
17    }
18
19    1 référence
20    public bool VerifyHashedPassword(string p_hashedPassword, string p_password)
21    {
22        PasswordVerificationResult result = _hasher.VerifyHashedPassword(
23            null!,
24            p_hashedPassword,
25            p_password
26        );
27        return result == PasswordVerificationResult.Success;
28    }
29 }
```

Le PasswordHasher d'ASP.NET Core Identity utilise un algorithme de dérivation de clé basé sur PBKDF2 pour produire le hachage du mot de passe. Le résultat de ce hachage est ensuite stocké sous une forme de chaîne de caractère dans la base de données.

Contrairement au chiffrement, le hachage est conçu comme une opération à sens unique : la valeur hachée ne permet pas de retrouver directement le mot de passe d'origine.

Essayer de déchiffrer un mot de passe haché est volontairement extrêmement coûteux et chronophage.

Lorsqu'on souhaite vérifier qu'un mot de passe donné est bien celui correspondant à celui sauvegardé, pour par exemple permettre à un utilisateur de se connecter.

On hach le mot de passe donné et le compare avec celui stocké.

Si ils sont identiques alors il s'agit du bon mot de passe.

De plus, avant de hacher un mot de passe, on utilise un Salt aléatoire sur ce dernier, cela permet de rendre plusieurs mot de passe haché utilisant le même mot de passe, différents les un des autres.

Exemple avec le mot de passe « Test1234& »

Id			Username		PasswordHash	
Filter			Filter		Filter...	
1	1	Test	AQAAAAIAAYagAAAAEGnHYRqix4qtlbV03/MrW2bfgoQDjL/8fe3trUIHIGoFlia/qQ5IE5s2U5OrTT6jg==			
2	2	Alexandre	AQAAAAIAAYagAAAAEH8MkXfODjNjpvB99jbPBRwLYsPTimqDI/ObdlnYawCNe4PHskR4x5nXNf5ixcTroA==			

Cela permet d'éviter que si l'un des mots de passe haché est compromis, alors tous les mots hachés utilisant le même mot de passe ne pourront pas être aussi compromis en utilisant la même valeur.

4. JWT et authentication des requêtes

Lorsqu'un utilisateur s'authentifie, il reçoit de la part du serveur un JWT (JSON Web Token).

Ce token sera sauvegardé sur le FrontEnd de l'utilisateur par le SeverRequestManager jusqu'à que l'utilisateur se déconnecte. Le token sera utilisé lors que l'utilisateur fera une requête au serveur impliquant des données utilisateurs.

L'objectif de ce JWT est que l'utilisateur puisse s'identifier lors de ces types de requêtes.

Lorsque le BackEnd reçoit une requête qui demande par exemple, à obtenir ou modifier des données utilisateur, il va pouvoir connaître grâce au JWT le UserId de l'utilisateur lui ayant envoyé la requête.

Permettant au BackEnd d'accéder aux données du bon utilisateur.

Cela permet aussi d'éviter que le client puisse simplement fournir un UserId arbitraire pour accéder aux données d'un autre utilisateur, car le BackEnd n'a pas besoin de demander au FrontEnd son UserId.

Code du UserProgressionController pour obtenir l'ID d'un utilisateur grâce au JWT :

```
228         if (IS_DEBUG_MODE_ON)
229             Console.WriteLine($"DEBUG: [{GetType().Name}] Starting to update the User's progression data.");
230
231         int userId = UserIdGetter.GetUserId(User);
```

Code de UserIdGetter :

```
3 namespace GaucheOuDroiteBackEnd.Tools
4 {
5     4 références
6     public static class UserIdGetter
7     {
8         /// <summary>
9         /// Returns the Id of a User.
10        ///
11        /// <para> It uses the AuthenticationToken to do so. </para>
12        /// </summary>
13        /// <param name="p_user"> You can only get the User if you are in a Class that inherits from ControllerBase. </param>
14        /// <returns></returns>
15        4 références
16        public static int GetUserId(ClaimsPrincipal p_user)
17        {
18            Claim? claim = p_user.FindFirst(ClaimTypes.NameIdentifier);
19
20            if (claim == null)
21                throw new Exception("Missing 'NameIdentifier' claim. The JWT may be configured incorrectly.");
22
23            return int.Parse(claim.Value);
24        }
25    }
26 }
```

5. Validation et protection des données reçues

A. Validation côté client

Dans certains cas, il est possible qu'en plus de la validation obligatoire du BackEnd, il y est une validation FrontEnd, c'est le cas ici, pour le système d'authentification.

L'intérêt de cette validation précoce est :

- d'avoir un retour immédiat lorsque l'utilisateur lance l'authentification, pas besoin d'attendre la réponse du serveur
- d'éviter de générer des requêtes inutiles

Un exemple de code du script Authenticator est disponible dans l'annexe à : « [4. Script Authenticator, méthode OnSendButtonPressed\(\)](#) ».

B. Validation côté serveur

Comme expliqué précédemment dans la sous-section « [A. Architecture multicouche](#) », les Controllers jouent un rôle dans la validation des données provenant du FrontEnd. Ils vérifient que les données attendues sont bien présentes.

Un exemple de code du script UserProgressionController est disponible dans l'annexe à : « [5. Script UserProgressionController, security checks](#) ».

C. Protection contre les injections

Afin de réduire le risque d'injection SQL, le projet utilise Entity Framework Core pour effectuer les opérations sur la base de données sans construire manuellement les requêtes SQL à partir des données fournies par l'utilisateur. Les valeurs sont ainsi traitées comme des paramètres plutôt que comme une partie de la requête.

Ainsi, un utilisateur malveillant ne peut pas essayer de modifier ces chaînes de caractère pour modifier à son avantage la base de données.

De plus, la validation des données reçues côté serveur constitue une protection complémentaire, mais ne remplace pas les mécanismes de paramétrage des requêtes.

6. Rate Limiting

Afin de limiter le risque d'envoi excessif de requêtes par un ou plusieurs clients, le système de Rate Limiting d'ASP.NET Core a été utilisé.

Il permet de manager le nombre maximal de requête qu'un utilisateur peut envoyer durant un certain laps de temps.

Pour ce projet il n'est possible que d'envoyer 5 requêtes durant une période de 10 secondes. Si une 6ème requête est envoyée durant ce laps de temps alors la requête sera automatiquement rejetée par le BackEnd, et un message d'erreur sera envoyé en FrontEnd.

Un exemple de code du script Program est disponible dans l'annexe à : « [6. Script Program, rate limiter setup](#) ».

7. Suppression des données utilisateur / RGPD

Afin de prendre en compte certains principes du RGPD (Règlement Général sur la Protection des Données), j'ai implémenté la possibilité pour l'utilisateur de supprimer son compte ainsi que les données qui lui sont associées.



Avant suppression du compte Player1

Id	#	Username	PasswordHash
1	1	Test	AQAAAAIAAYagAAAAEGnHYRqix...
2	2	Player1	AQAAAAIAAYagAAAAEN/xOOAh...
3	3	Player2	AQAAAAIAAYagAAAAEOJu2gMa...

Après suppression du compte Player1

Id	#	Username	PasswordHash
1	1	Test	AQAAAAIAAYagAAAAEGnHYRqix...
2	3	Player2	AQAAAAIAAYagAAAAEOJu2gMa...

VIII. Tests et validation

1. Stratégie et environnement de test

Les tests ont été réalisés tout au long du développement afin de vérifier le bon fonctionnement des fonctionnalités et de détecter les régressions pouvant être introduites par les modifications du projet.

La stratégie de test repose principalement sur des tests fonctionnels manuels. Après chaque modification importante, la fonctionnalité concernée était testée afin de vérifier que son comportement correspondait au fonctionnement attendu. Les fonctionnalités susceptibles d'être impactées par la modification étaient également vérifiées.

Cette approche a été choisie en raison de la taille du projet et du nombre relativement limité de fonctionnalités. Le jeu peut ainsi être parcouru et testé dans son ensemble en un temps raisonnable.

Les tests ont été réalisés dans plusieurs environnements.

Pendant le développement, les fonctionnalités étaient testées directement depuis l'environnement de développement.

Des tests supplémentaires étaient ensuite effectués sur les Builds du projet afin de vérifier que le fonctionnement observé dans l'environnement de développement était également présent dans la version distribuable.

2. Tests fonctionnels et techniques

Les tests fonctionnels portent sur les fonctionnalités du jeu. Chaque fonctionnalité était vérifiée à partir de plusieurs situations, notamment les cas d'utilisation normaux ainsi que les cas de saisies ou d'actions invalides.

Une attention particulière a été portée aux tests des systèmes de communication entre le FrontEnd et le BackEnd, ainsi qu'aux systèmes d'authentification.

Les requêtes envoyées par le Client ont été vérifiées afin de s'assurer que l'API recevait bien les données attendues et retournait correctement les résultats ou les erreurs associées.

Les mécanismes de sécurité ont également été testés. Cela comprend notamment les validations des données reçues par l'API, l'authentification à l'aide des JWT et le système de limitation du nombre de requêtes ont notamment été vérifiés.

Enfin, le fonctionnement du projet en condition réelle a été contrôlé depuis des Builds distribuables. Cela permettait notamment de détecter des problèmes qui ne seraient pas nécessairement visibles lorsque le projet était exécuté directement depuis l'environnement de développement.

C'est aussi via ces Builds que les testeurs ont pu donner leurs retours.

3. Tests de régression et validation utilisateur

Lorsqu'une modification était apportée au projet, je ne testais pas uniquement la fonctionnalité directement modifiée, mais également les fonctionnalités susceptibles d'être impactées par cette modification, et ce, afin de limiter les risques de régression.

Ces vérifications étaient particulièrement importantes lors de modification touchant à des scripts utilisés par d'autres scripts.

Lorsque je considérais les deux Builds testées comme étant valides, ces Builds étaient testées par plusieurs testeurs (personnes extérieures au projet).

Ces tests avaient un double objectif : détecter des bugs qui n'auraient pas été identifiés pendant le développement ou mes propres tests, et recueillir des retours concernant le jeu.

Les retours pertinents étaient ensuite analysés, listés, puis, lorsque nécessaire, intégrés au projet. Une nouvelle Build était alors produite et testée afin de vérifier les corrections apportées.

Exemple de retour :

- Test du jeu à mon père, les retours sont majoritairement positif.
Il demande cependant deux choses :
 - L'ajout de raccourcis lorsqu'on se Sign up ou Log in (Tab pour passer de Username à Password, et Entrée pour lancer la validation)
 - Faire qu'on puisse plus facilement savoir si on donne la bonne direction lorsqu'on est en jeu (dans Scene Level)

Cette démarche permettait d'effectuer plusieurs cycles de validation jusqu'à obtenir une version suffisamment stable pour poursuivre le développement.

4. Résultats et limites

Les différents cycles de tests ont permis de corriger les problèmes détectés au cours du développement et d'améliorer progressivement la stabilité mais aussi le résultat final du projet.

Les tests réalisés sur les Builds ainsi que les retours des personnes extérieures au projet ont également permis d'identifier des problèmes qui n'avaient pas été détectés pendant le développement.

Une des limites du projet concerne cependant l'absence de tests unitaires automatisés. Ce choix était volontaire et ne signifie pas que les tests unitaires ont été considérés comme inutiles.

Dans le contexte de ce projet, les fonctionnalités et donc le nombre de cas à tester sont relativement peu nombreux et peuvent être testés manuellement dans un temps raisonnable.

Le coût de mise en place et de maintenance d'une suite de tests automatisés m'a donc semblé supérieur au bénéfice qu'elle aurait apporté pendant la durée du projet.

Pour un projet plus complexe, cette décision aurait été différente.

Lorsque le nombre de fonctionnalités et de cas à vérifier augmente, les tests automatisés permettent notamment de réduire le temps nécessaire aux tests de régression et de garantir plus facilement le comportement de scripts déjà existants.

Par exemple, pour l'un de mes plus gros projets, j'ai décidé d'implémenter des tests unitaires, car le nombre de cas à vérifier est suffisamment important pour rendre les tests manuels trop longs.

IX. Déploiement et DevOps

1. Environnements Development / Production

Le projet distingue l'environnement de développement de l'environnement destiné à la distribution du logiciel.

Pendant le développement, le Client Unity et le Serveur ASP.NET Core sont exécutés depuis leurs environnements respectifs afin de permettre le développement, le débogage et les tests.

Lorsqu'une version est destinée à être distribuée, une Build du Client (FrontEnd) ainsi qu'une Build du serveur (BackEnd) sont préparées.

L'objectif est que cette version puisse fonctionner indépendamment de l'environnement de développement utilisé pour créer le projet.

La configuration du BackEnd est également séparée du code de l'application à l'aide des fichiers appsettings. Cela permet, par exemple, d'adapter les paramètres utilisés selon l'environnement d'exécution sans modifier directement le code source.

Les informations sensibles, telles que la clé utilisée pour la génération des JWT, ne sont pas directement intégrées dans le code source destiné à être versionné, mais stockées dans le fichier appsettings correspondant.

2. Préparation et déploiement du BackEnd

Après la première séance de test par des testeurs sur leur propre ordinateur, j'ai eu le retour comme quoi le serveur ne se lançait pas.

La cause de ce problème était que la version distribuée du BackEnd dépendait de composants nécessaires à l'exécution d'ASP.NET Core qui n'étaient pas présents sur les ordinateurs des testeurs.

J'ai donc modifié la procédure de publication du BackEnd afin d'intégrer les composants nécessaires à son exécution dans la version distribuée.

Cela permet son exécution sur n'importe quelle machine sans nécessité que l'utilisateur ait besoin d'installer d'autres fichiers que le .zip du projet.

Le BackEnd a donc été modifié afin de permettre son exécution indépendamment de l'environnement de développement. Les fichiers nécessaires au fonctionnement de l'API ainsi que sa configuration sont regroupés avec la version distribuée du projet.

Lors du déploiement, la base de données SQLite utilisée par l'API est également fournie avec les fichiers nécessaires au fonctionnement du serveur. Le serveur peut ainsi être lancé directement dans son environnement d'exécution sans avoir besoin d'ouvrir la solution dans Visual Studio.

Cette séparation entre l'environnement de développement et l'environnement d'exécution permet de se rapprocher des conditions réelles de déploiement d'une application.

Lors des premiers tests externes, un problème supplémentaire a été identifié concernant la configuration de la clé utilisée pour signer les JWT. Durant le développement, cette clé est stockée dans le fichier de configuration non versionné « appsettings.Production.json » afin d'éviter son exposition dans le dépôt GitHub.

Cependant, la version destinée aux testeurs devait pouvoir être exécutée directement sans nécessiter de configuration supplémentaire.

Une clé dédiée à cette version de test a donc été intégrée à sa configuration.

Ce choix constitue une limitation de la version distribuée : une personne ayant accès à cette clé pourrait potentiellement générer elle-même des JWT valides. Cette configuration n'est donc pas adaptée à une mise en production réelle.

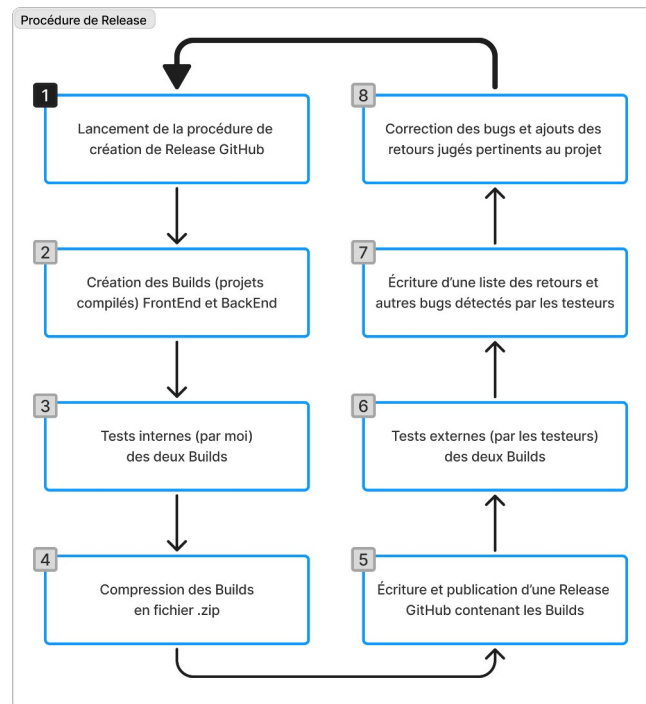
Dans le cas d'un déploiement commercial, une nouvelle clé secrète serait générée et conservée exclusivement sur le serveur. Elle ne serait jamais distribuée avec le Client. Le Client ne pourrait ainsi pas accéder à la clé permettant de signer les JWT.

Code du fichier appsettings.Production.json :

```
1 {
2   "JwtSettings": {
3     // Unlike during project development, here we are required to display the JWT encryption key in plain text.
4     // We are doing this in order to allow people who want to test the game, not to have to modify the project files (generate a key),
5     // they will be able to directly launch the server and the client and test the game.
6     //
7     // Even if we managed to make GitHub not see the key, a player could very well go and get the key directly from the game files.
8     //
9     // Having this key visible isn't necessarily a problem for this project.
10    // Because if ever the project becomes a commercial project, a new key will be generated and will only be accessible to a real server, not the client.
11    "Key": "f23964fd88606b0dd8963e645454cb0b804d65ac80501a5293e7242bbf3f84c3"
12  },
13  "Kestrel": {
14    "Endpoints": {
15      "Http": {
16        "Url": "http://localhost:9000"
17      }
18    }
19  }
20 }
```

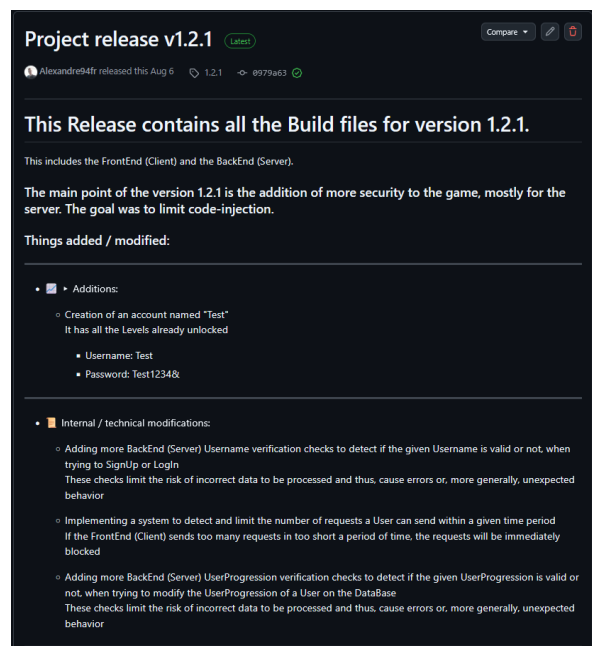
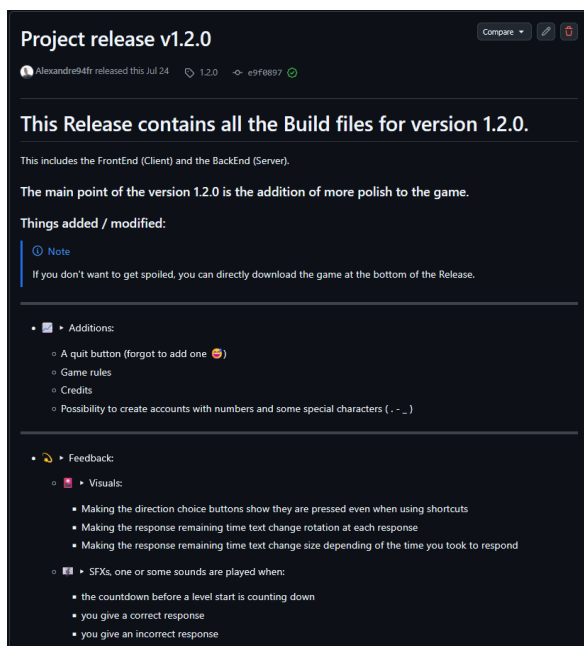
3. Builds et Releases

Lorsqu'une version du projet doit être déployée (Release), une procédure est lancée :



Cette boucle ne s'arrête que lorsque tous les problèmes détectés et priorisés ont été réglés, ainsi que lorsque tous les retours retenus ont été implémentés.

Chaque Release sur le dépôt GitHub du projet est accompagnée d'un patch note, détaillant les modifications apportées au projet :



4. Processus de mise en production et limites DevOps

Plusieurs pratiques associées à une démarche DevOps ont été intégrées au projet :

- versionnement du code avec GitHub
- utilisation de commits réguliers
- séparation des configurations et préparation du BackEnd pour son exécution indépendante de l'environnement de développement
- génération de Builds
- création de Releases versionnées

Cependant, aucune chaîne CI/CD entièrement automatisée n'a été mise en place. La génération des Builds, leur validation et leur distribution restent manuelles.

L'automatisation de ce processus aurait notamment pu être réalisée à l'aide d'une pipeline CI/CD permettant de générer automatiquement les Builds, d'exécuter des tests puis de produire une nouvelle Release.

Comme pour le choix de ne pas implémenter de UnitTest, ce choix est lié au contexte du projet et au temps disponible.

Le nombre relativement limité de fonctionnalités permettait de réaliser manuellement ces opérations dans un délai raisonnable. La mise en place et la maintenance d'une pipeline complète auraient représenté un investissement supplémentaire qui n'apportait pas selon moi, un bénéfice suffisant dans le cadre de ce projet.

Le processus manuel reste néanmoins structuré et reproductible. Chaque version officielle passe par les mêmes étapes de génération, de test et de validation avant d'être distribuée.

X. Bilan de projet

Malgré tous les différents challenges rencontrés et les 4 mois de travail que ce projet m'a demandé, je suis tout de même heureux d'avoir créé ce projet.

Il m'a permis d'apprendre des compétences que je ne connaissais, ou ne maîtrisais pas.

Je pense principalement ici, à la conception et création, en partant de presque zéro, d'une architecture Client / Serveur contenant une base de données relationnel.

Ces nouvelles compétences devrait mettre utile dans mon futur projet, car ce dernier comprend lui aussi une gestion de Clients et de Serveurs, ainsi qu'une base de données relationnel.

Et enfin, l'un des objectifs principales du projet a été atteint.

Je suis désormais plus rapidement capable d'identifier ma gauche et ma droite.

XI. Annexe

1. Script Commands.sql

Création des tables :

```
Commands.sql
1  -- SQLite
2
3  /*
4   DOCUMENTATION:
5
6   This file is used to run queries to the data base (DataBase.sqlite)
7   You can run specific commands (query) without having to uncomment it.
8   Just select the query and do Right click -> Run selected query
9  */
10
11
12  -- FOR DEVELOPMENT AND DEBUGGING:
13  -- DELETE FROM Users WHERE Username = 'Alexandre';
14  -- DELETE FROM UserProgressions WHERE UserId = 2;
15
16
17  PRAGMA foreign_keys = ON;
18
19
20  -- Code for GaucheOuDroite project:
21
22  -- You should prefer dropping the tables that are the last created.
23
24  DROP TABLE IF EXISTS LevelResponseTimeSteps;
25  DROP TABLE IF EXISTS UserProgressions;
26  DROP TABLE IF EXISTS Levels;
27  DROP TABLE IF EXISTS Users;
28
29
30  CREATE TABLE IF NOT EXISTS Users(
31      Id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
32
33      Username VARCHAR(50) UNIQUE NOT NULL,
34      PasswordHash VARCHAR(50) NOT NULL
35  );
```

```
37  CREATE TABLE IF NOT EXISTS Levels(
38      Id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
39
40      Name VARCHAR(50) NOT NULL,
41      Difficulty INTEGER NOT NULL CHECK (Difficulty >= 0),
42      IsInfinite NUMERIC NOT NULL,
43      ResponseSequence TEXT NOT NULL,
44      Star1MinimumScore INTEGER NOT NULL,
45      Star2MinimumScore INTEGER NOT NULL,
46      Star3MinimumScore INTEGER NOT NULL
47  );
48
49  CREATE TABLE IF NOT EXISTS UserProgressions(
50      Id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
51      UserId INTEGER NOT NULL,
52      LevelId INTEGER NOT NULL,
53
54      IsUnlocked NUMERIC NOT NULL,
55      BestScore INTEGER NOT NULL,
56
57      FOREIGN KEY(UserId) REFERENCES Users(Id),
58      FOREIGN KEY(LevelId) REFERENCES Levels(Id)
59  );
60
61  CREATE TABLE IF NOT EXISTS LevelResponseTimeSteps(
62      Id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
63      LevelId INTEGER NOT NULL,
64
65      MinimumCorrectResponses INTEGER NOT NULL,
66      MaximumResponseTimeInSeconds NUMERIC(15,5) NOT NULL,
67
68      FOREIGN KEY(LevelId) REFERENCES Levels(Id)
69  );
```

Ajout des données de niveaux : (tout le code n'est pas affiché ici, car trop long et répétitif)

```
88 -- All the Levels data will be create here
89 INSERT INTO Levels (
90     Name,
91     Difficulty,
92     IsInfinite,
93     ResponseSequence,
94     Star1MinimumScore,
95     Star2MinimumScore,
96     Star3MinimumScore
97 )
98 VALUES (
99     'Niveau 1',
100     0,
101     FALSE,
102     'LRLRRLLL',
103     3000,
104     5000,
105     8000
106 ),
107 (
108     'Niveau 2',
109     0,
110     FALSE,
111     'LLRLRRLL',
112     5000,
113     7500,
114     10000
115 ),
116 (
117     'Niveau 3',
118     1,
119     FALSE,
120     'LRLRLRLRLR',
121     7500,
122     10000,
123     12000
124 ),
125 (
126     'Niveau 4',
127     1,
128     FALSE,
129     'RLRRRLRLRL',
130     10000,
131     12000,
132     14000
133 ),
134 (
```

```
135     'Niveau 5',
136     2,
137     FALSE,
138     'RLRLRLRLRLRLRL',
139     12500,
140     15000,
141     19000
142 ),
143 (
144     'Niveau 6',
145     2,
146     FALSE,
147     'LRLRLRLRLRLRLRLRLRL',
148     15000,
149     20000,
150     24000
151 ),
152 (
153     'Infini',
154     3,
155     TRUE,
156     'L',
157     35000,
158     50000,
159     100000
160 );
161
162 -- No more manual UserProgressions adding
163 /*
164 > INSERT INTO UserProgressions ( ...
170 > )
171 > VALUES ( ...
198 > );
199 */
200
201 INSERT INTO LevelResponseTimeSteps (
202     LevelId,
203     MinimumCorrectResponses,
204     MaximumResponseTimeInSeconds
205 )
206 VALUES (
207     1,
208     0,
209     1
210 );
```

Avant d'arriver ici, vous étiez dans la sous-section : « [4. Modèle physique](#) ».

Mais vous pouvez directement lire la sous-section d'annexe suivante : « [2. Script DataBase.sqlite](#) »

2. Script DataBase.sqlite

Voici les données présentes dans la base de données par défaut :
(Après le lancement de Commands.sql + création du compte Test)

TABLES	
> LevelResponseTimeSteps	10
> Levels	7
> sqlite_sequence	4
> UserProgressions	7
> Users	1

LevelResponseTimeSteps

Id	LevelId	MinimumCorrectResponses	MaximumResponseTimeInSeconds
1	1	1	1
2	2	2	1
3	3	3	0.85
4	4	4	0.85
5	5	5	0.5
6	6	6	0.5
7	7	7	1
8	8	7	0.85
9	9	7	0.75
10	10	7	0.5

Levels

Id	Name	Difficulty	IsInfinite	ResponseSequence	Star1MinimumScore	Star2MinimumScore	Star3MinimumScore
1	Niveau 1	0	0	LRLRLLL	3000	5000	8000
2	Niveau 2	0	0	LLRLRLRL	5000	7500	10000
3	Niveau 3	1	0	LRLLRLRLRL	7500	10000	12000
4	Niveau 4	1	0	RLLRLRLRLRL	10000	12000	14000
5	Niveau 5	2	0	RLRLRLRLRLRLRL	12500	15000	19000
6	Niveau 6	2	0	LRRLRLRLRLRLRLRLRL	15000	20000	24000
7	Infini	3	1	L	35000	50000	100000

Users

Id	Username	PasswordHash
1	Test	AQAAAAIAAYagAAAAEGnHYRqix4qtibV03/...

UserProgressions

Id	UserId	LevelId	IsUnlocked	BestScore
1	1	1	1	8000
2	2	1	2	10000
3	3	1	3	12000
4	4	1	4	14000
5	5	1	5	19000
6	6	1	6	24000
7	7	1	7	58628

Avant d'arriver ici, vous étiez dans la sous-section : « [4. Modèle physique](#) ».

3. Script ServerRequestManager

```
25 références
11 public static class ServerRequestManager
12 {
13     7 références
14     public enum RequestType
15     {
16         /// <summary> To read BackEnd's values. </summary>
17         3 références
18         Get,
19         /// <summary> To read BackEnd header values. </summary>
20         // Head, // Not necessary for our project.
21         /// <summary> To create on the BackEnd. </summary>
22         1 référence
23         Post,
24         /// <summary> To replace completely BackEnd's values. </summary>
25         1 référence
26         Put,
27         /// <summary> To read BackEnd values. </summary>
28         // Create, // Not necessary for our project, Post is enough.
29         /// <summary> To delete BackEnd values. </summary>
30         1 référence
31         Delete
32     }
33
34     6 références
35     public const bool IS_DEBUG_MODE_ON = true;
36
37     /// <summary>
38     /// The URL the server uses to communicate with the Clients. <para></para>
39     ///
40     /// <list type="bullet">
41     /// <item><description> http://localhost:9888 For local http request. </description></item>
42     /// <item><description> https://localhost:9881 For local https request. </description></item>
43     /// </list>
44     /// </summary>
45     1 référence
46     public const string SERVER_URL = "http://localhost:9888";
47
48     8 références
49     public static string CLASS_NAME = typeof(ServerRequestManager).Name;
50
51     9 références
52     public static string AuthenticationToken = null;
```

```
10 références
53 public static string RequestResponseToString(UnityWebRequest p_request, bool p_mustDeserializedDownloadHandlerText = true)
54 {
55     return
56     $"Request response received. Data:\n" +
57     $"- Url: {p_request.url}\n" +
58     $"- Request type: {p_request.method}\n" +
59     $"- Result: {p_request.result}\n" +
60     $"- HTTP error: {p_request.error}\n" +
61     $"- Response code: {p_request.responseCode}\n" +
62     $"- Response body (formatted):\n" +
63     $"{{(p_mustDeserializedDownloadHandlerText ? JsonConvert.DeserializeObject(p_request.downloadHandler.text) : p_request.downloadHandler.text)}}";
64 }
65
66 /// <summary>
67 /// Sends an HTTP request to the BackEnd API and automatically handles:
68 /// <list type="bullet">
69 /// <item><description>Request creation (GET, POST, PUT, DELETE).</description></item>
70 /// <item><description>JSON serialization of the request body.</description></item>
71 /// <item><description>Authentication token injection when required.</description></item>
72 /// <item><description>Sending the request and waiting for the server response.</description></item>
73 /// <item><description>JSON deserialization of the response body.</description></item>
74 /// <item><description>Invoking success or error callbacks depending on the request result.</description></item>
75 /// </list>
76 ///
77 /// This method should be used as the main entry point for all communications
78 /// between the FrontEnd and the BackEnd.
79 ///
80 /// <para>
81 /// The server response body is automatically deserialized into the
82 /// specified <typeparamref name="TResponse"/> type.
83 /// </para>
84 ///
85 /// <para>
86 /// <b>BEWARE:</b> This method is a coroutine and must be started using
87 /// <see cref="MonoBehaviour.StartCoroutine(IEnumerator)">.
88 /// </para>
89 ///
90 /// <example>
91 /// Code example: Sending a login request:
92 /// <code>
93 /// StartCoroutine(ServerRequestManager.SendRequest<LoginResultDTO>(<
94 ///     "authentication/login",
95 ///     ServerRequestManager.RequestType.Post,
96 ///     new LoginDTO
97 ///     {
98 ///         Username = username,
99 ///         Password = password
100 ///     },
101 ///     p_isAuthenticationTokenNeeded: false,
```

```

102 ///
103 ///     result =>
104 ///     {
105 ///         Debug.Log($"Successfully log in as {result.Username}");
106 ///         ServerRequestManager.AuthenticationToken = result.Token;
107 ///     },
108 ///
109 ///     request =>
110 ///     {
111 ///         Debug.LogError(
112 ///             $"Failed to log in. Request failed.\n" +
113 ///             ServerRequestManager.RequestResponseToString(request)
114 ///         );
115 ///     }
116 /// };
117 /// </code>
118 /// </example>
119 /// </summary>
120 ///
121 ///
122 /// <typeparam name="TResponse"> Type expected from the server response body. </typeparam>
123 /// <param name="p_route"> API route relative to "/api/". Example: "authentication/log-in". </param>
124 /// <param name="p_requestType"> HTTP request type to send. </param>
125 /// <param name="p_requestBody"> Object to serialize as JSON and send to the server. Can be null for requests that do not require a body. </param>
126 /// <param name="p_isAuthenticationTokenNeeded"> Indicates whether the request requires an AuthenticationToken.
127 /// If true, the value stored in <see cref="AuthenticationToken"/> will be added to the request headers.
128 /// Be sure that <see cref="AuthenticationToken"/> as been set before passing true.
129 /// A warning will be print out if <see cref="AuthenticationToken"/> is equal to null or "". </param>
130 /// <param name="p_onSuccess"> Callback invoked when the request succeeds. Receives the deserialized server response. </param>
131 /// <param name="p_onError"> Callback invoked when the request fails. Receives the original UnityWebRequest for error inspection. </param>
132 ///
133 /// <returns> IEnumerator used by Unity's coroutine system. </returns>
134 6 references
135 public static IEnumerator SendRequest<TResponse>(
136     string p_route,
137     RequestType p_requestType,
138     object p_requestBody,
139     bool p_isAuthenticationTokenNeeded,
140     Action<TResponse> p_onSuccess,
141     Action<UnityWebRequest> p_onError
142 )
143 {
144     // -- Creating the request -- //
145     string url = $"{SERVER_URL}/api/{p_route}";
146     string requestType = p_requestType.ToString().ToUpper();
147     if (IS_DEBUG_MODE_ON)
148         Debug.Log($"DEBUG: [{CLASS_NAME}] Creating a request.\n{requestType} request for '{url}'.");
149
150     UnityWebRequest request = new(
151         url,
152         requestType
153     );
154     // -- Preparing request's body (content) -- //
155     if (IS_DEBUG_MODE_ON)
156         Debug.Log($"DEBUG: [{CLASS_NAME}] Preparing request's body (content).\n{requestType} request for '{url}'.");
157     // Converting the raw body in Json
158     string requestBodyJson = JsonConvert.SerializeObject(p_requestBody);
159     byte[] body = Encoding.UTF8.GetBytes(requestBodyJson);
160     // Will store the data when we send the request to the server
161     request.uploadHandler = new UploadHandlerRaw(body);
162     // Will store the data when the server will respond
163     request.downloadHandler = new DownloadHandlerBuffer();
164     request.SetRequestHeader(
165         "Content-Type", // To tell the Backend (server) that we stored a body (content) in Json inside the request, so he can detect it.
166         "application/json"
167     );
168     if (p_isAuthenticationTokenNeeded)
169     {
170         if (IS_DEBUG_MODE_ON)
171             Debug.Log($"DEBUG: [{CLASS_NAME}] Adding to request's body (content) an AuthenticationToken.\n{requestType} request for '{url}'.");
172         if (string.IsNullOrEmpty(AuthenticationToken))
173         {
174             Debug.LogWarning(
175                 $"WARNING: [{CLASS_NAME}] Tried to send a server request containing an AuthenticationToken when none have been saved. " +
176                 $"Try logging in first. Request canceled. Returning."
177             );
178             yield break;
179         }
180         request.SetRequestHeader(
181             "Authorization", // To tell the Backend (server) that we stored an AuthenticationToken inside the request, so he can detect it.
182             $"Bearer {AuthenticationToken}"
183         );
184     }
185     // -- Sending the request and waiting for a response -- //
186     if (IS_DEBUG_MODE_ON)
187         Debug.Log($"DEBUG: [{CLASS_NAME}] Sending the request to the server.\n{requestType} request for '{url}'.");

```

```

285     yield return request.SendWebRequest();
286
287     // -- Handling server response -- //
288
289     if (IS_DEBUG_MODE_ON)
290         Debug.Log($"DEBUG: [{CLASS_NAME}] Handling server's request response.\n{requestType} request for '{url}'.");
291
292     // Getting and converting the server response body
293     TResponse responseBody;
294
295     try
296     {
297         responseBody = JsonConvert.DeserializeObject<TResponse>(request.downloadHandler.text);
298     }
299     catch (Exception exception)
300     {
301         // Note:
302         // This catch will be called if 'request.downloadHandler.text' is a pure string, not an object serialized into a string, like expected
303         // But the error can still be caused by something else, check the warning below to gain more information
304
305         Debug.LogWarning(
306             $"WARNING: [{CLASS_NAME}] Caught an error while trying to deserialize the server response content into '{typeof(TResponse)}' type. Returning." +
307             $"\\nREQUEST CONTENT:\\n{RequestResponseToString(request, false)}\\n" +
308             $"\\nERROR:\\n{exception}"
309         );
310
311         yield break;
312     }
313
314     if (IS_DEBUG_MODE_ON)
315         Debug.Log($"DEBUG: [{CLASS_NAME}] {RequestResponseToString(request)}");
316
317     // Invoking the right Action depending of the success of the request
318     if (request.result == UnityWebRequest.Result.Success)
319     {
320         p_onSuccess?.Invoke(responseBody);
321     }
322     else
323     {
324         p_onError?.Invoke(request);
325     }
326 }
327
328 }

```

Avant d’arriver ici, vous étiez dans la sous-section : « [4. Communication avec l'API BackEnd](#) ».

4. Script Authenticator, méthode OnSendButtonPressed()

```

0 références
238 public void OnSendButtonPressed()
239 {
240     #region Checking if the given authentication values are correct
241
242     AuthenticationProperties.AuthenticationErrorReasons errorReason;
243
244     if (!IsUsernameValid(_username, out errorReason))
245     {
246         string errorMessage = GetErrorMessage(errorReason);
247
248         // Modifying the UI to tell the player, that his username is incorrect.
249         DisplayFeedback(...);
250
251         _authenticationInformationGameObject.SetActive(true);
252
253         if (_isDebugModeOn)
254             Debug.Log($"DEBUG: [{GetType().Name}] The given username '{_username}' is not valid, reason: {errorMessage} Returning");
255
256         return;
257     }
258
259     if (!IsPasswordValid(_password, out errorReason))
260     {
261         string errorMessage = GetErrorMessage(errorReason);
262
263         // Modifying the UI to tell the player, that his password is incorrect.
264         DisplayFeedback(...);
265
266         _authenticationInformationGameObject.SetActive(true);
267
268         if (_isDebugModeOn)
269             Debug.Log($"DEBUG: [{GetType().Name}] The given password '{new string('*', _password.Length)}' is not valid, reason: {errorMessage} Returning");
270
271         return;
272     }
273
274     #endregion
275
276     // Modifying the UI to tell the player, that his username and password are correct and that we sent a request to the server.
277     DisplayFeedback(...);
278
279     if (_isDebugModeOn)
280         Debug.Log($"DEBUG: [{GetType().Name}] The given username and password are correct, sending them to the Authentication API.");
281
282     // Sending these values to the correct Authentication API (in BackEnd)
283     // Will also disable some UI buttons to avoid creating bugs.
284     // The UI buttons will be interactive again when the server will respond to the request.
285     StartCoroutine(SendAuthenticationRequestToServer(_authenticationMode));
286 }
287
288 }

```

Avant d’arriver ici, vous étiez dans la sous-section : « [A. Validation côté client](#) ».

5. Script UserProgressionController, security checks

```
146 #region == Security checks ==
147
148 if (IS_DEBUG_MODE_ON)
149     Console.WriteLine($"DEBUG: [{GetType().Name}] Starting to check if the given UserProgressionDTO is valid.");
150
151 // Getting all Levels' properties.
152 // Will be used to verify received LevelProgression data.
153 List<Level> levels = await _levelService.GetAllLevelsAsync();
154
155 if (p_userProgressionDTO == null)
156 {
157     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO argument is null.");
158 }
159
160
161 if (p_userProgressionDTO.LevelProgressions == null)
162 {
163     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions argument is null.");
164 }
165
166
167 if (p_userProgressionDTO.LevelProgressions.Count <= 0)
168 {
169     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions argument is an empty list.");
170 }
171
172
173 if (p_userProgressionDTO.LevelProgressions.Count != levels.Count)
174 {
175     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions argument is a list that doesn't have the same count as the list of Levels in the game.");
176 }
177
178
179 for (int i = 0; i < p_userProgressionDTO.LevelProgressions.Count; i++)
180 {
181     if (p_userProgressionDTO.LevelProgressions[i].Id < 0)
182     {
183         return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].Id argument is under 0.");
184     }
185
186     if (p_userProgressionDTO.LevelProgressions[i].LevelId < 0)
187     {
188         return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].LevelId argument is under 0.");
189     }
190
191     if (p_userProgressionDTO.LevelProgressions[i].LevelId > levels.Count)
192     {
193         return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].LevelId argument is superior to the number of Levels in the game.");
194     }
195 }
196
197
198 if (p_userProgressionDTO.LevelProgressions[i].IsUnlocked && !IsPreviousLevelUnlocked(p_userProgressionDTO.LevelProgressions, i))
199 {
200     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].IsUnlocked (LevelId: {p_userProgressionDTO.LevelProgressions[i].LevelId}) argument is not valid.");
201 }
202
203
204 if (p_userProgressionDTO.LevelProgressions[i].BestScore < 0)
205 {
206     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].BestScore argument is under 0.");
207 }
208
209 // Note: If the Level is not infinite you can't have more score than the 'Star3MinimumScore'
210 if (levels[i].IsInfinite == false && p_userProgressionDTO.LevelProgressions[i].BestScore > levels[i].Star3MinimumScore)
211 {
212     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].BestScore argument is superior to the maximal score for a finite Level.");
213 }
214
215 // Note: The DataBase stores the BestScore using an 'int', we don't want a User to break the DataBase by passing a value bigger than an 'int'.
216 if (levels[i].IsInfinite == true && p_userProgressionDTO.LevelProgressions[i].BestScore > INFINITE_LEVEL_MAXIMAL_SCORE)
217 {
218     return ReturnBadRequestResult(ref apiResponseDTO, $"The given UserProgressionDTO.LevelProgressions[{i}].BestScore argument is superior to the maximal score for an infinite Level.");
219 }
220
221 if (IS_DEBUG_MODE_ON)
222     Console.WriteLine($"DEBUG: [{GetType().Name}] The given UserProgressionDTO is valid.");
223
224 #endregion
```

Avant d'arriver ici, vous étiez dans la sous-section : « [B. Validation côté serveur](#) ».

6. Script Program, rate limiter setup

```
96 builder.Services.AddRateLimiter(options =>
97 {
98     options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(httpContext =>
99     {
100         return RateLimitPartition.GetFixedWindowLimiter(
101             // We use the IP address to detect who is sending the request
102             //
103             // Note:
104             // Creating partitions on client IP addresses makes the app vulnerable to Denial of Service Attacks which employ IP Source Address Spoofing.
105             // For more information, see BCP 38 RFC 2827 Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing.
106             partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
107
108             factory: _ => new FixedWindowRateLimiterOptions
109             {
110                 PermitLimit = 5,
111                 Window = TimeSpan.FromSeconds(10),
112                 QueueProcessingOrder = QueueProcessingOrder.OldestFirst,
113                 QueueLimit = 0
114             }
115         );
116     });
117 };
118
119 // Creating a custom rejection handling logic
120 options.OnRejected = async (context, cancellationToken) =>
121 {
122     Console.WriteLine($"WARNING: [{SCRIPT_NAME}] Rate limit exceeded for IP: {context.HttpContext.Connection.RemoteIpAddress}");
123
124     context.HttpContext.Response.StatusCode = StatusCodes.Status429TooManyRequests;
125     context.HttpContext.Response.Headers.RetryAfter = "10";
126
127     ApiResponseDTO apiResponseDTO = new()
128     {
129         HasSucceeded = false,
130         ErrorMessage = "Rate limit exceeded. Please try again a little later."
131     };
132
133     await context.HttpContext.Response.WriteAsync(ObjectToStringFormatter.ObjectToString(apiResponseDTO), cancellationToken);
134 };
135
136 };
137
```

Avant d'arriver ici, vous étiez dans la sous-section : « [6. Rate Limiting](#) ».